Learning the Difficulty of Derivative Problems

Lecture notes written while building `calc-adaptive-assessment`

Dylan Costa

Contents

0 The problem: difficulty is a latent variable **1**

1 Generating problems: random trees **2**

2 Differentiation as structural recursion **5**

3 Features: what the model is allowed to see **8**

4 Bradley–Terry: the first actual learning **11**

5 Evaluation: what does "good" mean? **15**

6 Linear Bradley–Terry on features: generalizing to new problems **17**

7 Collecting real labels **20**

8 Real labels: what the model learned about you **23**

9 Gradient boosting: gradient descent in function space **27**

10 Active learning: which question to ask next **30**

11 A neural network that reads the expression tree **33**

12 Explaining a prediction: Shapley values **37**

0 The problem: difficulty is a latent variable

We want a function $\hat{\delta} : \mathcal{X} \to \mathbb{R}$ on the set $\mathcal{X}$ of derivative problems such that $\hat{\delta}(x) > \hat{\delta}(x')$ when an experienced instructor would call x harder. An adaptive exam (an optional Section 14) needs exactly this number for every problem it might ask. Since $\mathcal{X}$ is generated by a program, it is effectively infinite, and we will label only a tiny subset. So $\hat{\delta}$ has to *generalize*.

Why comparisons, not ratings. Nobody observes difficulty. The obvious label, a rating $r \in \{1, \dots, 5\}$, is a poor instrument. A realistic model of a rater is $r_t(x) = g_t(\delta(x)) + \varepsilon$, where g_t is an *unknown, drifting, increasing* function (the scale shifts during a session, and each rating is anchored on the last one). Regressing on r_t learns the moving target $g_t \circ \delta$. A comparison "is

x harder than x'?" sees g_t only through its monotonicity, so g_t drops out (Thurstone's *law of comparative judgment*, 1927). Our labels will be comparisons, modeled by *Bradley–Terry*:

$$\mathbb{P}(x \succ x') = \sigma\big(\delta(x) - \delta(x')\big), \qquad \sigma(t) = \frac{1}{1+e^{-t}}. \tag{1}$$

Equation (1) is invariant under $\delta \mapsto \delta + c$, so δ is identified only up to a constant. The *scale*, however, is fixed by the link: a gap of 1 means a 73% win rate. The same equation trains the reward models used in RLHF for large language models, and its online version is Elo.

Learning = picking a function from a class. Estimating one free number δ_i per labeled problem is classical statistics (Section 4). It says nothing about an unseen problem. Machine learning instead picks f_θ from a class $\mathcal{F} = \{f_\theta : \theta \in \Theta\}$ that can be evaluated anywhere, by *empirical risk minimization*:

$$\hat\theta \in \arg\min_\theta \underbrace{\frac{1}{n}\sum_{k=1}^{n} \ell(\theta; z_k)}_{\widehat{R}_n(\theta)} + \lambda\,\Omega(\theta), \qquad \text{hoping that} \qquad R(\theta) = \mathbb{E}_{z\sim\mathcal{D}}\,\ell(\theta; z) \text{ is small too.}$$

Here $z = (x, x', y)$ is a labeled pair, ℓ is the negative log-likelihood of (1), and Ω is a regularizer. The gap $R - \widehat{R}_n$ (*generalization error*) grows with the richness of $\mathcal{F}$ relative to n. With a few hundred labels from one instructor, n is small, and that tension runs through the project. Every design decision lands in one of four slots:

slot	question	sections
distribution $\mathcal{D}$	which problems and pairs do we see?	1, 7, 10 (active learning)
class $\mathcal{F}$	what can f_θ see and express?	2, 3 (features), 6, 9, 11
loss ℓ, regularizer Ω	what counts as a good fit?	4, 6
optimization	how do we find $\hat\theta$?	4, 6, 9, 11

Roadmap. 1 generator · 2 step-tracing differentiator · 3 features · 4 Bradley–Terry MLE · 5 evaluation · 6 linear BT on features · 7 labeling tool · 8 learning curves · 9 boosting · 10 active learning · 11 tree neural nets · 12 Shapley values · 13 a guessing game, and how different people judge difficulty (multi-rater models) · 14 (optional, with real students) an adaptive quiz (IRT).

How the repo maps to these notes. The package is `src/calcdiff/`, tests are in `tests/`, and every number and figure here is produced by `scripts/figs_stepNN.py` and pulled in with `\input`, so the notes cannot silently disagree with the code.

1 Generating problems: random trees

Every guarantee in learning is relative to a distribution, and ours comes from a program *we* write. Its choices decide which problems get labeled and where the model will be forced to extrapolate. So we treat the generator as a probability distribution, not as "code that makes random problems".

Problems are trees, not functions. Let $\mathcal{X}$ be the set of expression trees: internal nodes $+, \times, \div, (\cdot)^q, \sin, \ldots, \sqrt{\ }$, leaves x or rationals. Let $\pi : \mathcal{X} \to \{\text{functions}\}$ map a tree to the function it denotes. π is far from injective: $\frac{x}{x+1}$ and $x(x+1)^{-1}$ are the same function but different problems (quotient rule vs. product and chain rule). Difficulty is a function on $\mathcal{X}$, not on $\pi(\mathcal{X})$. That is why

we keep our own tree type: SymPy would store `x/(x+1)` as `Mul(x, Pow(x+1, -1))`, destroying the distinction. Each node type is a frozen dataclass:

From src/calcdiff/expr.py

```
@dataclass(frozen=True)
class Div(Node):
    num: Node
    den: Node
```

and `to_sympy` realizes π, so SymPy remains the ground truth for the *mathematics*.

A probabilistic grammar. A PCFG is a context-free grammar plus, for each nonterminal, a probability distribution over its productions. Sampling expands nonterminals top-down, and the probability of a tree is the product of the probabilities of the choices made. Ours has one interesting nonterminal F:

$$F \;\to\; \underbrace{c\,x^{n}}_{\text{LEAF } p_{\text{leaf}}} \;\mid\; \underbrace{g(F) \text{ or } g(F{+}c)}_{\text{FN } p_{\text{fn}}} \;\mid\; \underbrace{(F{+}c)^{n}}_{\text{POWF } p_{\text{powf}}} \;\mid\; \underbrace{F \cdot F}_{\text{PROD } p_{\text{prod}}} \;\mid\; \underbrace{F/F}_{\text{QUOT } p_{\text{quot}}} \quad ,$$

with the whole problem a sum of 1–3 such terms. The code is a literal transcription. Each production is an `if`, and each F on a right-hand side is a recursive call:

From src/calcdiff/generate/grammar.py

```
def _F(self, d: int) -> Node:
    ...
    rule = self._choice(RULES, self._rule_p)      # pick a production
    if rule == "FN":
        g = self._choice(self._fn_names, self._fn_p)
        inner = self._F(d + 1)                    # the F inside g(F)
        ...
        return Fn(g, inner)
    if rule == "PROD":
        return Mul((self._F(d + 1), self._F(d + 1)))
```

Every random draw goes through `_choice` or `_bernoulli`, which also add $\log \mathbb{P}(\text{draw})$ to a running total. So each sample carries its exact $\log p(x)$ (the grammar is unambiguous, so $p(\text{tree}) = p(\text{derivation})$). A test checks this by hand for x^2.

Does the sampler terminate? Branching processes. Each PROD/QUOT spawns two new F's, so termination is not obvious. Counting only F-nodes, each has $\xi \in \{0, 1, 2\}$ children independently, with $p_0 = p_{\text{leaf}}$, $p_1 = p_{\text{fn}} + p_{\text{powf}}$, $p_2 = p_{\text{prod}} + p_{\text{quot}}$. That is a Galton–Watson process with offspring generating function $f(s) = p_0 + p_1 s + p_2 s^2$ and mean $m = f'(1) = p_1 + 2p_2$.

Theorem 1.1. *The probability q that the tree is finite is the smallest root of $f(s) = s$ in $[0, 1]$. For our quadratic f the roots are 1 and p_0/p_2, so $q = \min(1, p_0/p_2)$, and $q = 1$ iff $m \le 1$.*

Sketch. $q_d := \mathbb{P}(\text{extinct by depth } d) = f^{\circ d}(0)$ increases to q, which is therefore a fixed point. For any fixed point $r \ge 0$, monotonicity gives $f^{\circ d}(0) \le f^{\circ d}(r) = r$, so q is the smallest. □

Our defaults have $m = 0.70$ (subcritical). The code also caps the depth at 4, which makes every rule setting consistent. How large are the trees? Let N be the total number of F-nodes. The root's subtrees are i.i.d. copies of the whole tree, so $N \overset{d}{=} 1 + \sum_{j \le \xi} N^{(j)}$, which gives $\mathbb{E}N = 1 + m\mathbb{E}N$,

i.e. $\mathbb{E}N = 1/(1-m)$. Better still, with $G(s) = \mathbb{E}s^N$ the same decomposition says $G = s\,f(G)$, and *Lagrange inversion* reads off every coefficient:

$$\mathbb{P}(N = n) = [s^n]G = \tfrac{1}{n}[w^{n-1}]f(w)^n = \tfrac{1}{n}\,\mathbb{P}(S_n = n-1), \qquad S_n = \xi_1 + \cdots + \xi_n, \tag{2}$$

which is the Otter–Dwass formula. Figure 1(a) checks it against 50,000 untruncated samples. The empirical mean is 3.314 versus $1/(1-m) = 3.333$, and the largest tree had 99 F-nodes. The tail decays like $n^{-3/2}\rho^n$ with $\rho \to 1$ as $m \to 1$, which is the practical reason for the depth cap.

Rejection: conditioning on "an instructor would write this". Raw draws include $x^2 \cdot \frac{x^2}{x^2}$, $6x^2 + x^2$, and (a case the real labels of Section 8 exposed) stacked coefficients like $3 \cdot 9x\,(x^2+9)^5$, where a term's coefficient multiplies a product that already carries one. `is_natural` defines an acceptance set A, and `sample()` redraws until it lands in A. The stacked-coefficient rule flattens each product and allows at most one constant, in front, and at most one monomial in x:

From src/calcdiff/generate/grammar.py

```
case Mul(fs):
    ...
    flat = _flatten_product(node)                    # a(bc) -> [a, b, c]
    consts = [k for k, f in enumerate(flat) if isinstance(f, Const)]
    if consts not in ([], [0]):                       # 3 * 9x * u, u * 7x
        return False
    if sum(_is_monomial(f) for f in flat) > 1:        # 9x * sin(x) * x
        return False
```

Tightening A changes the accepted distribution, so every number in these notes was regenerated after the fix. If $X_t \sim p$ i.i.d. and τ is the first hit, then $\mathbb{P}(X_\tau = x) = \sum_t (1-p(A))^{t-1}p(x)\mathbf{1}_A(x) = p(x \mid A)$. Here $p(A) \approx 0.686$. The stored $\log p$ is unconditional, and the missing $-\log p(A)$ cancels in any ratio. Table 1 shows accepted draws. Note that N tracks difficulty only roughly; making that precise is the whole project.

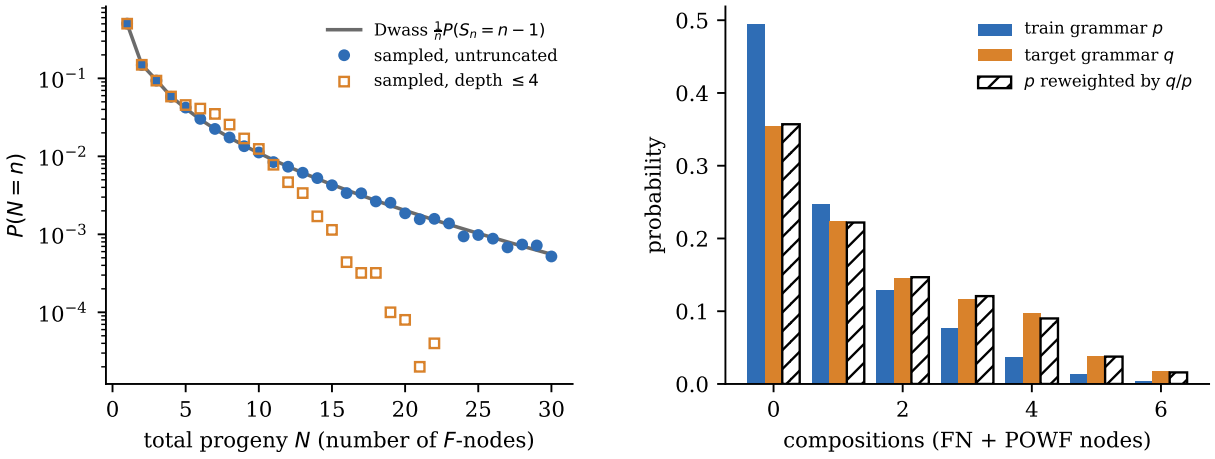
N	$\log p$	problem: find $\frac{d}{dx}$ of
1	-3.9	x^3
2	-11.2	$8\tan\left(x^2\right)$
3	-12.6	$\ln\left(\ln\left(x^{1/2}\right)\right)$
4	-14.1	$\sqrt{x^3}x^{-1}$
5	-22.0	$\ln\left(\ln\left(-7x^2\right)\right) + x^2 + x^3$
6	-34.9	$9x^2\left(\cos\left(\left(x^{-1}-2\right)^2\right)-2\right)^3 - 1$
7	-28.2	$6\left(\frac{x^2}{\sqrt{x^{3/2}}x^2} + 8\right)^2$
8	-31.8	$x\left(x^{-1}\left(x^{1/3}+8\right)^3 + 7\right)^2 + x$

Table 1: Accepted samples (`seed=2026`), one per derivation size N.

Covariate shift. If we train on $x \sim p$ but deploy on $x \sim q$ (say, a harder exam bank), then $R_q(\theta) = \mathbb{E}_p[w(x)\ell(\theta; x)]$ with $w = q/p$, provided $q \ll p$. In general w must be estimated. For us it is exact, and it has a nice form.

Proposition 1.2. *If grammars p, q differ only in rule probabilities, then* $\log \frac{q(x)}{p(x)} = \sum_r c_r(x) \log \frac{q_r}{p_r}$, *where $c_r(x)$ counts free uses of rule r. So the grammar family is an exponential family with sufficient statistic $c(x)$.*

(Proof: every non-rule choice has the same probability under p and q and cancels.) `log_likelihood_ratio` implements exactly this sum. Figure 1(b) reweights p-samples to imitate a grammar q with more compositions. The estimated mean number of compositions is 1.594 versus the true 1.612, and the effective sample size is $\mathrm{ESS} = 1/\sum \bar{w}_i^2 = 14{,}540$ out of 20,000. Lesson: hard problems are rare under p ($\mathbb{P}_p(\geq 5 \text{ compositions}) = 0.018$ vs. 0.065 under q), so a model trained on p extrapolates there. Reweighting cannot fix missing *support*, only the wrong shape.



(a) (b)

Figure 1: (a) Law of the number of F-nodes: formula (2) against the sampler. (b) Covariate shift between two grammars, corrected by the exact likelihood ratio of Proposition 1.2.

Exercises.

Exercise 1.1. Show $\operatorname{Var} N = \sigma^2/(1-m)^3$ with $\sigma^2 = \operatorname{Var} \xi$. (The test `test_branching_process_mean_progeny` uses this to set its tolerance.)

Exercise 1.2. For i.i.d. $x_i \sim p$, show $\mathrm{ESS}/M \to 1/\mathbb{E}_p[w^2] = 1/(1+\chi^2(q\|p))$, and use Proposition 1.2 to write $\mathbb{E}_p[w^2]$ as a moment generating function of $c(x)$.

2 Differentiation as structural recursion

Before learning anything, we can *measure* part of what makes a problem hard: the procedure a student must carry out. Step 2 builds a differentiator that does what a student does, one textbook rule at a time, and logs every rule it uses.

One clause per constructor. Trees are built from a few constructors (`Const`, `Var`, `Pow`, `Add`, `Mul`, `Div`, `Fn`). A function on trees defined by one clause per constructor, calling itself only on immediate subtrees, is well defined by *structural induction*: the recursion terminates because every call is on a

strictly smaller tree. Differentiation is exactly such a function. Python's `match` statement lets the code read like the definition:

From src/calcdiff/solver_trace.py

```
def _d(node, tr, depth, chain):
    def log(rule):                          # record "rule applied to node"
        tr.steps.append(Step(rule, node, depth, chain))
    def d(sub, *, through_chain=False):  # recurse one level down
        return _d(sub, tr, depth + 1, chain + through_chain)

    match node:
        case Pow(base, n):                          # (u^n)' = n u^(n-1) u'
            log("POWER")
            outer = _mul(Const(n), _pow(base, n - 1))
            if isinstance(base, Var):
                return outer
            log("CHAIN")
            return _mul(outer, d(base, through_chain=True))
        case Mul(factors):                          # (f1...fk)' = sum_i f_i' prod_{j!=i} f_j
            ...
            log("PRODUCT")
            terms = [_mul(*rest[:i], d(f), *rest[i+1:]) for i, f in enumerate(rest)]
            return _mul(c, _add(*terms))
        case Fn(name, arg):                         # g(u)' = g'(u) u'
            log(f"FN_{name}")
            outer = _fn_derivative(name, arg)
            ...
```

Read it as math. `d(sub)` is $D(\text{sub})$ one level deeper in the recursion. The flag `through_chain=True` marks that we passed *inside* a composition, which is how `chain_depth` counts nested chain rules. `log` is a side effect that does not change the value computed. The helpers `_mul`, `_add` do only the cleanup a person does automatically ($1 \cdot u = u$, $u + 0 = u$, multiply the constants together), so the output is the derivative *as a student would first write it*, unsimplified.

Why the rules compose: dual numbers. Let $\mathbb{R}[\varepsilon] = \mathbb{R}[t]/(t^2)$, so $\varepsilon^2 = 0$. For a polynomial, Taylor's theorem is exact: $p(a + b\varepsilon) = p(a) + b\,p'(a)\varepsilon$. Use this to *define* $g(a + b\varepsilon) := g(a) + b\,g'(a)\varepsilon$ for $g = \sin, \exp, \ldots$. Then

$$(a + a'\varepsilon)(b + b'\varepsilon) = ab + (a'b + ab')\varepsilon, \qquad g(u + u'\varepsilon) = g(u) + g'(u)\,u'\varepsilon,$$

so the product rule is ring multiplication and the chain rule is composition. Formally, evaluating a tree at $x + \varepsilon$ is a homomorphism from expressions into $\mathbb{R}[\varepsilon]$, and the ε-coefficient is the derivative. Our recursion is the symbolic shadow of this, and the numeric version is *forward-mode automatic differentiation*. Every neural network in this course is trained by computing gradients this way, or rather by its transpose, *reverse mode* (backpropagation), which we will meet in Section 11.

The trace is a measurement. For $x^2 \sin\left(e^{3x}\right)$ the code returns $2x \sin\left(e^{3x}\right) + 3x^2 \cos\left(e^{3x}\right) e^{3x}$ and the trace in Table 2. *depth* is how many rule applications enclose this one (roughly, how much a student must hold in working memory), and *chain* is how many compositions we are inside. From each trace we get a vector of counts $c(x) \in \mathbb{N}^{\text{rules}}$, the total `n_steps`, `max_depth`, and `max_chain_depth`. These will become features in Section 3.

Over 3,000 generated problems, 0.71 need the chain rule, 0.17 the product rule, and 0.12 the quotient rule. 0.31 have nested chains (`max_chain_depth` ≥ 2). The number of steps has Spearman

#	depth	chain	rule	applied to
1	0	0	`PRODUCT`	$x^2 \sin\left(e^{3x}\right)$
2	1	0	`POWER`	x^2
3	1	0	`FN_sin`	$\sin\left(e^{3x}\right)$
4	1	0	`CHAIN`	$\sin\left(e^{3x}\right)$
5	2	1	`FN_exp`	e^{3x}
6	2	1	`CHAIN`	e^{3x}
7	3	2	`CONST_MULT`	$3x$
8	4	2	`VAR`	x

Table 2: Trace of `derive` on $x^2 \sin\left(e^{3x}\right)$, in the order the recursion visits nodes.

correlation 0.91 with the grammar's derivation size N. That is unsurprising, since both mostly measure size. Size is not difficulty, though: `n_steps` counts $\frac{d}{dx}5x^3$ and the inner $\frac{d}{dx}3x$ of $\sin(e^{3x})$ the same, even though only the second is buried two compositions deep. Telling them apart is what features and learning are for.

Expression swell. Figure 2(a) plots the size of the derivative against the size of the problem. A least-squares fit on log-log axes has slope 1.51: derivatives grow faster than linearly. The product rule is to blame. A product of k factors of total size n has a derivative with k terms, each containing all k factors (one differentiated), so size $\Theta(kn)$, which is quadratic when the factors are small. The median derivative is 1.60× the problem's size, and the worst is 5.4×. Swell is a real component of difficulty, since a student has to write it all down without dropping a term.

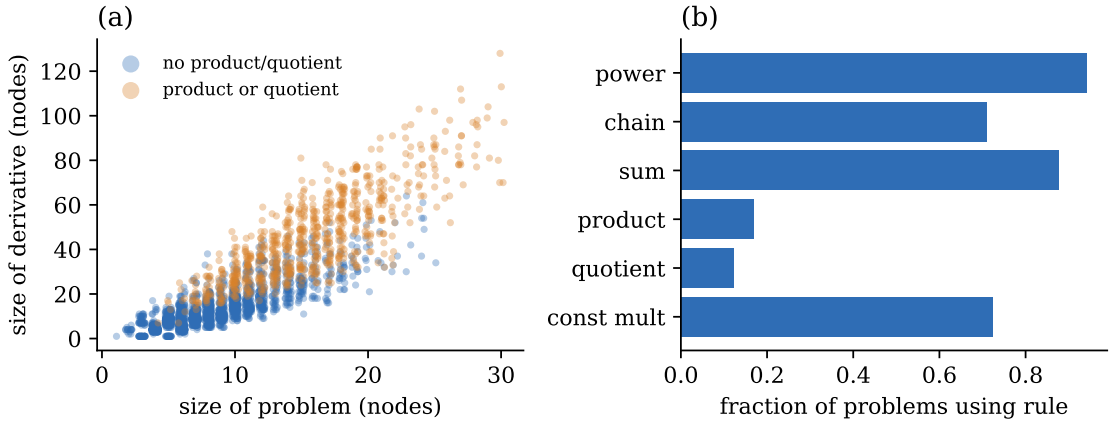


Figure 2: (a) Expression swell: derivative size vs. problem size (jittered). (b) How often each rule is needed.

Testing it: the identity theorem as a test oracle. How do we know `derive` is right? Comparing with `sympy.diff` symbolically (simplify$(a - b) = 0$) is slow and can fail to prove true identities. Instead, `numerically_equal` in `src/calcdiff/check.py` evaluates both at random points z near the positive real axis, slightly off it in $\mathbb{C}$:

From `src/calcdiff/check.py`

```
z = rng.uniform(0.3, 2.5, n) + 1j * rng.uniform(-0.1, 0.1, n)
```

```
va, vb = fa(z), fb(z)                          # both sides, vectorized
ok = np.isfinite(va) & np.isfinite(vb)         # skip poles / overflow
return np.all(np.abs(va[ok] - vb[ok]) <= rtol * (1 + np.abs(vb[ok])))
```

Why is this a proof-quality test? Both sides are holomorphic on a connected open set U (off poles and branch cuts). If $h = a - b \not\equiv 0$ on U, its zeros are isolated by the identity theorem, hence countable, hence of measure zero. A point drawn from a continuous distribution on U detects $h \neq 0$ with probability 1 (in exact arithmetic, with a tolerance in floating point). Complex points step off the branch cuts of ln and $\sqrt{\ }$, where real points can land on the wrong side. Both sides use the same principal branches, and $\frac{d}{dz}\operatorname{Log} z = 1/z$ holds off the cut.

`test_matches_sympy_on_generated_problems` runs this on 500 problems. In the statistics run, 2,989 of 3,000 agree, none disagree, and 11 are *inconclusive*. Those are problems like $e^{(x+9)^3} \approx e^{1000}$ that overflow double precision everywhere. Arbitrary precision does not rescue them cheaply, since cos of a number near 10^{400} needs about 400 digits of π just to reduce the argument mod 2π. So the checker honestly returns `None`. The generator found something too: $e^{(x^2+9)^5}$ is legal but a bit absurd. We will see whether the instructor's labels penalize such problems.

Exercises.

Exercise 2.1. Using only $\varepsilon^2 = 0$, derive the quotient rule from $(u + u'\varepsilon)/(v + v'\varepsilon)$. (Why is $v + v'\varepsilon$ invertible in $\mathbb{R}[\varepsilon]$ exactly when $v \neq 0$?)

Exercise 2.2. Let $P = f_1 \cdots f_k$ with $|f_i| = s$ nodes each, and suppose each $|Df_i| \leq \alpha s$. Bound $|DP|$ as produced by `_mul`/`_add`, and show the bound is $\Theta(k^2 s)$. What does this predict for the log-log slope in Figure 2(a), and why might the measured 1.51 be smaller?

Exercise 2.3. The random-point test can be fooled when $a - b$ vanishes on an open set, which cannot happen for a nonzero holomorphic function on a connected domain. Give a pair of expressions whose difference is nonzero but vanishes on the entire sampling region $\{0.3 < \operatorname{Re} z < 2.5,\ |\operatorname{Im} z| < 0.1\}$ of the code. (Hint: think about $\sqrt{z^2}$ and the branch cut, then about where that region sits.) Does this break our argument?

3 Features: what the model is allowed to see

Every model until Section 11 sees a problem only through a *feature map*

$$\varphi : \mathcal{X} \to \mathbb{R}^d, \qquad d = 17,$$

and a linear model predicts $f_\theta(x) = \theta \cdot \varphi(x)$. So the hypothesis class is $\mathcal{F} = \{\theta \cdot \varphi : \theta \in \mathbb{R}^d\}$, a d-dimensional vector space of functions on $\mathcal{X}$, spanned by the coordinate functions $\varphi_1, \ldots, \varphi_d$. This is the cleanest way to say what choosing features means: **you are choosing the subspace in which the answer must live**. If the true difficulty δ is not close to that span, no amount of data will help (*bias*). If the span is large relative to the data, the fit is noisy (*variance*). Section 8 makes that trade-off quantitative.

The code. `featurize` assembles $\varphi(x)$ from three sources: the tree itself, the trace from Section 2, and the derivative the student has to write down.

From `src/calcdiff/features.py`

```python
def featurize(tree: Node) -> np.ndarray:
    d, tr = derive(tree)                    # Step 2: derivative + trace
    c = tr.counts()                         # Counter: rule name -> times used
    pows = [n for n in walk(tree) if isinstance(n, Pow)]
    f = {
        "size": size(tree),
        "deriv_size": size(d),              # expression swell
        "n_chain": c["CHAIN"],
        "max_chain_depth": tr.max_chain_depth,
        "n_quotient": c["QUOTIENT"],
        "n_neg_exp": sum(p.exp < 0 for p in pows),
        "chain_in_product": _chains_under_product(tree, False),
        ...                                 # 17 features in all
    }
    return np.array([f[name] for name in FEATURE_NAMES], dtype=float)
```

The dictionary-then-array pattern is deliberate. `FEATURE_NAMES` fixes the coordinate order once, so column j of every matrix in the project means the same thing. For our running example $x^2 \sin(e^{3x})$, Table 3 lists every coordinate. Check them against the trace in Table 2.

size	8	n_const_mult	1
depth	5	n_trig	1
n_terms	1	n_exp	1
deriv_size	22	n_ln	0
n_power	1	n_sqrt	0
n_chain	2	n_neg_exp	0
max_chain_depth	2	n_frac_exp	0
n_product	1	chain_in_product	2
n_quotient	0		

Table 3: $\varphi(x)$ for $x = x^2 \sin(e^{3x})$.

φ is a quotient map, and that's a modeling claim. Any function of the form $\theta \cdot \varphi$ is constant on the fibers $\varphi^{-1}(v)$. So every model we build until Section 11 asserts that *problems with equal features are equally hard*. Among 3,000 generated problems there are only 2,095 distinct feature vectors. The largest fiber has 37 problems (e.g. $4x^4$ and $3x^4$: fine, these really are equally hard). But $\cos(x^2)$ and $\sin(x^2)$ also collide, because `n_trig` lumps sin, cos and tan together. An instructor might well say the cosine one is harder (that minus sign!). If so, our model *cannot* learn it: the information was discarded before learning began. Deciding which distinctions to keep is the art of feature engineering, and it is where your teaching experience enters the model.

Interactions: what "linear" cannot do. A linear model is *additive*: the effect of one more chain rule is the same θ_{chain} whether or not there is a product around it. If you believe a chain rule *inside* a product rule is harder than the two separately (the student must hold $u'v + uv'$ open while doing a chain rule inside it), the model can only express that through a feature equal to that interaction:

From src/calcdiff/features.py

```python
def _chains_under_product(node: Node, under: bool) -> int:
    here = int(under and _is_chain(node))      # a chain rule with a product above it?
    under = under or (isinstance(node, (Mul, Div)) and _has_two_nonconst(node))
    return here + sum(_chains_under_product(ch, under) for ch in node.children())
```

This is another structural recursion, like Section 2, but now the flag `under` is *inherited* downward: once we pass through a product or quotient (a genuine one, not $5 \cdot u$), every chain rule below it counts. Hand-built interactions do not scale (there are $\binom{d}{2}$ pairs). Gradient-boosted trees (Section 9) and neural networks (Section 11) are, in large part, machines for discovering interactions automatically.

Geometry of the feature matrix. Stack the features of n problems into $\Phi \in \mathbb{R}^{n \times d}$. Figure 3(a) shows the correlations over 3,000 problems. Several features nearly duplicate each other: *n_chain* and *max_chain_depth* have correlation 0.92, and `size` with `n_power` has 0.78. The spectrum of the correlation matrix, panel (b), shows how close Φ is to rank-deficient. The eigenvalues run from 6.78 down to 0.012 (condition number ≈ 557), and the two smallest eigenvectors are essentially "`size` $\approx$ a combination of the rule counts" and "`deriv_size` $\approx$ a combination of the rule counts".

Why care? For least squares with noise variance σ^2, $\mathrm{Cov}(\hat{\theta}) = \sigma^2(\Phi^\top \Phi)^{-1}$. Along the eigenvector v_i of $\Phi^\top \Phi$ with eigenvalue λ_i, $\mathrm{Var}(v_i \cdot \hat{\theta}) = \sigma^2/\lambda_i$. Directions with tiny λ_i are directions in which the data barely constrain θ. The individual weights on `size` and the rule counts will swing wildly from sample to sample, even though their combined prediction is stable. (Section 4 shows that Bradley–Terry has exactly this structure, with $\Phi^\top \Phi$ replaced by a weighted version.) Two standard fixes, both coming in Section 6: drop redundant features, or add a ridge penalty $\frac{\lambda}{2}\|\theta\|^2$, which replaces λ_i by $\lambda_i + \lambda$ and caps every variance at σ^2/λ.

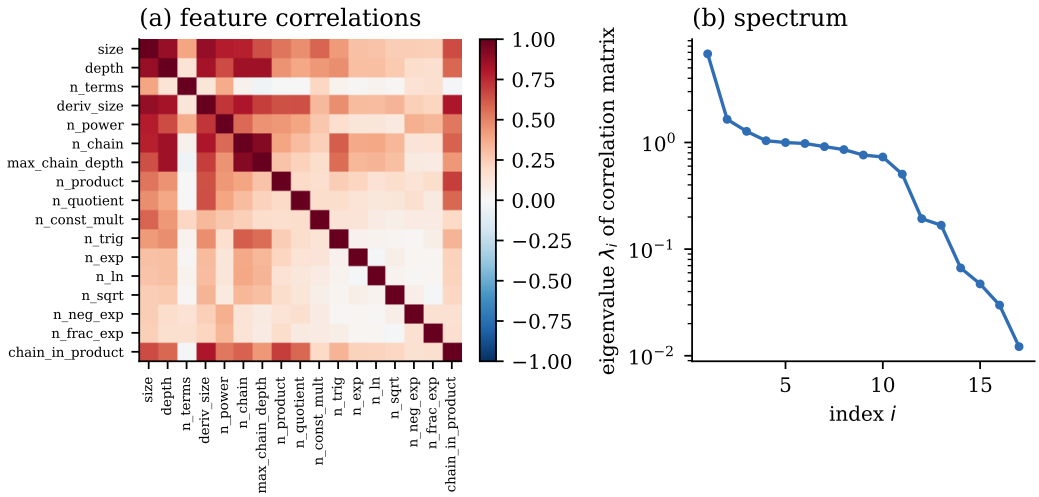


Figure 3: (a) Correlations among the 17 features on 3,000 generated problems. (b) Eigenvalues of the correlation matrix (log scale). The last few are what make individual weights unstable.

Standardization, and the first appearance of leakage. A ridge penalty is not invariant under rescaling a feature. Measuring `size` in nodes (mean ≈ 10) while `n_quotient` is mostly 0 or 1 would quietly penalize them very differently (Exercise 3.1). The standard fix is $z_j = (\varphi_j - \mu_j)/s_j$:

From src/calcdiff/features.py

```
class Standardizer:
    def fit(self, X):
        self.mean_ = X.mean(axis=0)
        sd = X.std(axis=0)
        self.std_ = np.where(sd > 0, sd, 1.0)   # a constant column stays constant
        return self
```

```
    def transform(self, X):
        return (X - self.mean_) / self.std_
```

The split into `fit` and `transform` is the important design point. μ and s are *parameters estimated from data*, so they must be fit on training problems only and then applied, frozen, to test problems. Computing them on everything lets test data influence training, which is called *leakage*. It is a small effect here, and a fatal one in many real projects. Section 5 is about this.

Exercises.

Exercise 3.1. Show that minimizing $\widehat{R}(\theta) + \frac{\lambda}{2}\|\theta\|^2$ with feature φ_j replaced by $c\,\varphi_j$ is equivalent to the original problem with the penalty on θ_j changed from $\frac{\lambda}{2}\theta_j^2$ to $\frac{\lambda}{2c^2}\theta_j^2$. Which features does an unstandardized ridge penalize most?

Exercise 3.2. Propose one feature that would separate $\cos(x^2)$ from $\sin(x^2)$, and one that captures "the answer needs simplification". Write each as a structural recursion like `_chains_under_product`. Which of your features are invariant under reordering the terms of a sum? Should they be?

4 Bradley–Terry: the first actual learning

Now we fit the model (1) to comparisons. In this section each problem $i \in \{1, \dots, n\}$ gets its own free parameter θ_i, with no features yet. This is the classical estimation problem, and almost everything about the feature-based model of Section 6 already shows up here.

The likelihood, as logistic regression on differences. Comparison k involves items (i_k, j_k) with label $y_k = 1$ if i_k was judged harder. Put $s_k = 2y_k - 1 \in \{\pm 1\}$ and $t_k = \theta_{i_k} - \theta_{j_k}$. Since $1 - \sigma(t) = \sigma(-t)$, both outcomes have likelihood $\sigma(s_k t_k)$, and the negative log-likelihood is

$$L(\theta) = \sum_k \log(1 + e^{-s_k t_k}) + \tfrac{\lambda}{2}\|\theta\|^2, \qquad t = D\theta, \tag{3}$$

where D is the $m \times n$ *design matrix* with rows $e_{i_k} - e_{j_k}$. That is all logistic regression is: a linear map followed by $u \mapsto \log(1 + e^{-u})$ (*softplus*). In Section 6 we will simply replace the rows $e_i - e_j$ by $\varphi(x_i) - \varphi(x_j)$, and every line of code below carries over unchanged.

From `src/calcdiff/models/bt.py`

```
def one_hot_design(pairs, n_items):
    D = np.zeros((len(pairs), n_items))
    rows = np.arange(len(pairs))
    D[rows, pairs[:, 0]] = 1.0            # +e_i
    D[rows, pairs[:, 1]] = -1.0           # -e_j
    return D

def nll(theta, D, y, l2=0.0):
    t = D @ theta
    # cross-entropy: softplus(t) - y t
    return float((np.logaddexp(0.0, t) - y * t).sum() + 0.5 * l2 * theta @ theta)
```

The code uses the equivalent form $\log(1 + e^{t_k}) - y_k t_k$ (the *cross-entropy* between y_k and $\sigma(t_k)$). For $y_k \in \{0, 1\}$ it equals the summand in (3), and it also makes sense for $y_k = \frac{1}{2}$, which Section 7 needs for ties. `np.logaddexp(0, u)` computes $\log(e^0 + e^u) = \log(1 + e^u)$ without ever forming e^u. The naive `np.log(1 + np.exp(u))` overflows to `inf` for $u > 710$, which happens as soon as a fit grows confident. Numerical care like this is a large part of practical ML.

Gradient and Hessian. With $p = \sigma(D\theta)$ and $W = \operatorname{diag}(p(1-p))$, a short computation using $\sigma' = \sigma(1-\sigma)$ gives

$$\nabla L = D^\top(p - y) + \lambda\theta, \qquad \nabla^2 L = D^\top W D + \lambda I.$$

The gradient has a nice reading: each comparison pushes its two items apart by "predicted minus observed".

From src/calcdiff/models/bt.py

```
def grad_hess(theta, D, y, l2=0.0):
    p = expit(D @ theta)                                    # sigma, elementwise
    g = D.T @ (p - y) + l2 * theta
    H = (D * (p * (1 - p))[:, None]).T @ D + l2 * np.eye(len(theta))
    return g, H
```

`D * w[:, None]` scales row k of D by w_k via broadcasting, so the last line is $D^\top W D$ without building the $m \times m$ matrix W. A test (`test_gradient_matches_finite_differences`) checks ∇L against central differences. Always do this when you hand-derive a gradient.

The Hessian is a graph Laplacian. Expand $D^\top W D = \sum_k w_k (e_{i_k} - e_{j_k})(e_{i_k} - e_{j_k})^\top$. This is the Laplacian L_w of the *comparison graph* G on the items, with an edge of weight $w_k = p_k(1-p_k)$ for each comparison. Its quadratic form is

$$v^\top L_w v = \sum_k w_k \, (v_{i_k} - v_{j_k})^2 \;\geq 0.$$

So L is convex, and $v^\top L_w v = 0$ iff v is constant on each connected component of G (all $w_k > 0$). Two consequences:

- **Identifiability.** If G is connected, $\ker L_w = \operatorname{span}(\mathbf{1})$: the data pin down θ up to exactly one additive constant, as promised in Section 0. If G has two components, each one floats independently, and comparing items across them is meaningless.
- **The gauge is preserved automatically.** For singular H, the least-squares solver used in `fit_newton` returns the minimum-norm solution of $H\,\Delta = -g$, which lies in $\operatorname{range}(H) = \mathbf{1}^\perp$. Starting at $\theta = 0$, every iterate satisfies $\sum_i \theta_i = 0$.

The test `test_hessian_is_weighted_graph_laplacian` checks all of this numerically: symmetric, rows summing to 0, off-diagonals ≤ 0, and exactly one zero eigenvalue.

Does a maximizer exist? Zermelo's condition. Convexity gives uniqueness (on $\mathbf{1}^\perp$), but not existence. Suppose a set S of items never *lost* to an item outside S. Then adding $c > 0$ to every θ_i, $i \in S$, increases every cross-term $s_k t_k$, so the loss strictly decreases for all c and the infimum is not attained. The converse holds as well:

Theorem 4.1 (Zermelo 1929; Ford 1957)**.** *With $\lambda = 0$, L has a minimizer (unique on $\mathbf{1}^\perp$) iff the directed graph with an edge winner→loser for each comparison is strongly connected.*

From src/calcdiff/models/bt.py

```
def mle_exists(pairs, y, n_items):
    win = np.where(y[:, None] == 1, pairs, pairs[:, ::-1])   # rows (winner, loser)
    G = coo_matrix((np.ones(len(win)), (win[:, 0], win[:, 1])), shape=(n_items, n_items))
    n_comp, _ = connected_components(G, directed=True, connection="strong")
    return n_comp == 1
```

This is not a corner case. In the very first test I wrote, with 30 items and about 40 comparisons each, one item (true $\delta = -3.3$) lost all of its games, and Newton happily drove θ for that item past -27. Worse, it then reported *convergence*: along the escape ray the gradient tends to 0, so a gradient-based stopping rule is fooled. The test `test_perfect_separation_diverges_without_ridge` pins this behavior down. With $n = 200$ items, Experiment C below found a finite MLE in a fraction 0.0 of the datasets with 32 comparisons per item. The more items, the more chances that *some* item goes undefeated or winless.

The fix: ridge = MAP with a Gaussian prior. Put a prior $\theta_i \overset{\text{iid}}{\sim} N(0, 1/\lambda)$. The negative log-posterior is (3) with the $\frac{\lambda}{2}\|\theta\|^2$ term, which is strictly convex and coercive, so the *maximum a posteriori* (MAP) estimate always exists and is unique. We use $\lambda = 0.1$ (prior sd 3.2 logits, very weak) from now on. A pleasant fact (Exercise 4.1): because every row of D sums to zero, the ridge optimum satisfies $\sum_i \theta_i = 0$ exactly. In the experiment below, $|\sum_i \hat{\theta}_i| = 1e-14$.

Optimization: Newton vs. gradient descent. Newton's method minimizes the local quadratic model $L(\theta) + g^\top \Delta + \frac{1}{2}\Delta^\top H \Delta$, giving $\Delta = -H^{-1}g$:

From `src/calcdiff/models/bt.py`

```
for it in range(1, max_iter + 1):
    g, H = grad_hess(theta, D, y, l2)
    step = np.linalg.lstsq(H, -g, rcond=None)[0]     # solve H step = -g
    decrement = float(-g @ step)                      # Newton decrement g^T H^-1 g
    if decrement / 2 < tol:                           # ~ L(theta) - L*
        return BTFit(theta, it - 1, True, hist, H)
    t = 1.0
    while nll(theta + t * step, D, y, l2) > hist[-1] - 0.25 * t * decrement:
        t *= 0.5                                      # backtrack until enough decrease
    theta = theta + t * step
```

Two details. The quantity $\lambda^2 = g^\top H^{-1} g$ (the *Newton decrement*) is the decrease the quadratic model predicts. Half of it estimates the optimality gap $L(\theta) - L^*$, which makes it a principled, scale-free stopping rule. And the `while` loop is an Armijo backtracking line search: far from the optimum the quadratic model can be wrong, so we halve the step until the actual decrease is at least a quarter of the predicted one. Near the optimum $t = 1$ is accepted and convergence is *quadratic* (the number of correct digits doubles each step).

Gradient descent instead takes $\theta \leftarrow \theta - \eta \nabla L$ with $\eta = 1/\beta$, where $\beta = \|D\|_2^2/4 + \lambda \geq \lambda_{\max}(H)$ since $p(1-p) \leq \frac{1}{4}$. For a μ-strongly convex, β-smooth function it converges *linearly*: $L(\theta_t) - L^* \leq (1 - \mu/\beta)^t (L(\theta_0) - L^*)$. Figure 4(b) shows the difference: Newton needs 6 iterations, while gradient descent needs 220 to reach a gap of 10^{-10} (the Hessian at the optimum has condition number 113). Why does all of deep learning use gradient methods anyway? A Newton step costs $O(d^3)$, and $d = 10^9$ is common. Here $d = 200$, so Newton is the right tool.

Uncertainty: Fisher information and effective resistance. Near the optimum the log-posterior is approximately quadratic, so the posterior is approximately $N(\hat{\theta}, H^{-1})$ (the *Laplace approximation*). The frequentist twin is asymptotic normality, $\hat{\theta} \approx N(\theta^*, \mathcal{I}^{-1})$, with the Fisher information $\mathcal{I} = \mathbb{E}[D^\top W D]$. For a difference of two items,

$$\operatorname{Var}(\hat{\theta}_i - \hat{\theta}_j) \approx (e_i - e_j)^\top L_w^+ (e_i - e_j) = R_{\text{eff}}(i, j),$$

the *effective resistance* between i and j in the electrical network with conductance w_k on each comparison edge. That gives two design lessons for later. Many parallel paths of comparisons lower

the variance, just as parallel resistors do. And each comparison's conductance is $w = \sigma'(\delta_i - \delta_j) \le \frac{1}{4}$, maximal for *evenly matched* pairs and exponentially small for lopsided ones. Comparing x^2 with a monster problem teaches us almost nothing. This is the seed of active learning (Section 10).

Remark 4.2 (a bug that was really a lesson). My first version of Experiment C predicted an RMSE of 0.33 from the Fisher information, while the simulation achieved 0.24. The theory was beating reality! The culprit was the direction $\mathbf{1}$. The data have no curvature there, so H^{-1} assigns it the pure prior variance $1/\lambda = 10$, which adds $10/n$ to every diagonal entry. But $\hat{\theta} \perp \mathbf{1}$ exactly, so there is never any error in that direction. The right covariance is $PH^{-1}P$ with $P = I - \frac{1}{n}\mathbf{1}\mathbf{1}^\top$ (`covariance(centered=True)`), and with it theory and simulation agree. The naive mean standard error was 0.50, and the corrected one is 0.44.

Experiments on a planted truth. `src/calcdiff/synthetic.py` defines a simulated instructor: $\delta(x) = w \cdot \varphi(x) + \tau\varepsilon(x)$, with hand-picked weights w (e.g. 0.55 logits per quotient rule) and an idiosyncratic part $\tau = 0.5$ that the features cannot see. It then answers comparisons by sampling from (1). My first weights gave δ a spread of 10 logits, and only about 10% of random datasets with 40 comparisons per item satisfied Zermelo's condition. The extremes simply never lost. Scaling the weights down gave sd 1.45 and range $[-2.7, 5.0]$. On 200 problems with 4,000 random comparisons (40 per item), Figure 4:

(a) **Recovery.** RMSE 0.48 logits, Spearman $\rho = 0.929$ with the truth, and 94% of the nominal 95% intervals $\hat{\theta}_i \pm 1.96\,\mathrm{se}_i$ contain δ_i. Note the error bars widen at the extremes: lopsided comparisons carry little information.

(b) **Optimization.** Newton converges quadratically, while gradient descent converges linearly.

(c) **Rate.** RMSE against comparisons per item k, averaged over 5 datasets. The log-log slope is -0.51, matching the $k^{-1/2}$ predicted by $\mathcal{I} \propto k$. The dashed Fisher curve is computed from the truth with no fitting, and it lands on the simulation (0.25 vs. 0.24 at $k = 128$). At $k = 2$ the prior dominates: the estimates barely leave 0, so the RMSE (1.39) is close to the spread of δ itself (1.45).

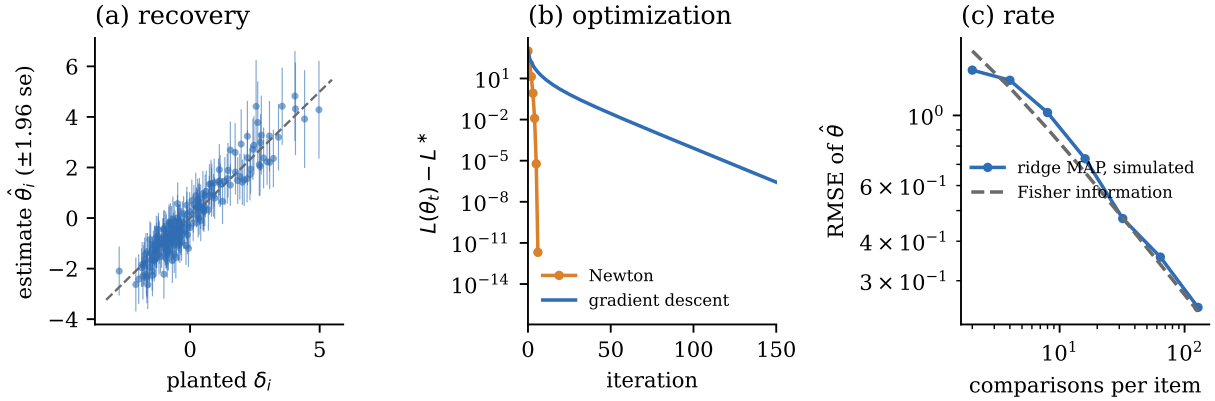


Figure 4: Bradley–Terry on synthetic data. (a) Estimates vs. planted difficulty with 95% intervals. (b) Optimality gap per iteration. (c) Error vs. comparisons per item, against the Fisher-information prediction.

What this model cannot do. Item-level BT has nothing to say about problem $n + 1$: it has no parameter for it. And it needs on the order of $k \approx 40$ comparisons *per problem* for ±0.5-logit

accuracy, which is far beyond any instructor's patience for a large bank. Section 6 replaces e_i by $\varphi(x_i)$. The number of parameters drops from n to $d = 17$, and information is *shared*: a comparison involving one quotient problem teaches the model about all quotient problems.

Exercises.

Exercise 4.1. Show that for one-hot D and $\lambda > 0$, the minimizer of (3) satisfies $\mathbf{1}^\top \hat{\theta} = 0$. (Hint: $D\mathbf{1} = 0$. Take the inner product of the stationarity equation with $\mathbf{1}$.)

Exercise 4.2. Items $1, \ldots, n$ compared only along a path (1 vs. 2, 2 vs. 3, ...), each pair r times, all with $w = \frac{1}{4}$. Compute $\mathrm{Var}(\hat{\theta}_1 - \hat{\theta}_n)$ via effective resistance, and compare with the complete graph using the same total number of comparisons. What does this say about how to schedule an instructor's comparisons?

Exercise 4.3. Prove the "only if" direction of Zermelo's theorem carefully. Then explain why it implies that the maximum likelihood estimate fails to exist with probability tending to 1 if $n \to \infty$ with k fixed.

5 Evaluation: what does "good" mean?

A model is only as trustworthy as the experiment that evaluates it. This section builds that experiment before we have a model worth evaluating, so that we can't tune the experiment to flatter the model.

Hold out problems, not comparisons. The deployed model will score problems *nobody has compared*. So the target is

$$R = \mathbb{E}_{x,x' \sim \mathcal{D}}\, \mathbb{E}_{y|x,x'}\, \ell\big(\hat{\delta}(x) - \hat{\delta}(x'),\, y\big), \qquad x, x' \text{ independent of the training problems.}$$

The independent unit, the thing that is exchangeable, is the *problem*. If we split comparisons at random, the same problems sit on both sides of the split, and we measure interpolation among problems we have already seen. That is *leakage*: information about the test set reaching the model through the back door. Table 4 makes it concrete. Item-level BT from Section 4 looks excellent on new comparisons among problems it has seen, and is a coin flip on unseen problems, because it has nothing to say about them.

model	evaluated on	accuracy	oracle ceiling
item-level BT	new pairs, seen problems	0.733	0.760
item-level BT	unseen problems	0.500	0.774
feature BT	new pairs, seen problems	0.733	0.760
feature BT	unseen problems	0.747	0.774

Table 4: The same two models under two evaluation protocols (synthetic instructor; 200 training problems, 200 unseen).

Metrics.

- *Pairwise accuracy*: the fraction of decided comparisons ordered correctly by the scores. If every pair were labeled by the true order, accuracy would be exactly $(1+\tau)/2$ with τ Kendall's tau (Exercise 5.1).
- *Log-loss*: the mean cross-entropy $\frac{1}{m}\sum_k[\log(1+e^{t_k}) - y_k t_k]$. It is a *proper scoring rule*: its expectation is minimized by reporting the true probability (Gibbs' inequality). So it rewards calibrated confidence, not just the right order. We use it to *tune* and accuracy to *report*.
- *Rank correlation* with the truth (Spearman's ρ, Kendall's τ). This is only available on synthetic data.

From src/calcdiff/evaluate.py

```python
def pairwise_accuracy(scores, pairs, y, weights=None):
    decided = y != 0.5                              # skip "about the same"
    diff = scores[pairs[:, 0]] - scores[pairs[:, 1]]
    tied = np.abs(diff) < 1e-9                      # equal scores, up to float dust
    credit = np.where(tied, 0.5, ((diff > 0) == (y == 1)).astype(float))
    w = np.ones(len(y)) if weights is None else weights
    w = w * decided
    return float((w * credit).sum() / w.sum())
```

Two conventions matter. A model that refuses to choose (equal scores) gets half credit, so a constant model scores *exactly* 0.5. And the tolerance `1e-9` is not cosmetic. My first run reported 0.492 for item-level BT on unseen problems, where every score is 0. The θ's were $\pm 10^{-17}$ rather than exactly zero, so ties were being broken at random. The optional `weights` argument is for the bootstrap below.

The noise ceiling. Labels are random: even a perfect model is right on a comparison only with probability $\max(p, 1-p)$, where $p = \sigma(\delta_i - \delta_j)$. So no model can beat $C = \mathbb{E}\max(p, 1-p)$ (`bayes_ceiling`), and how close we get matters more than the raw number. Figure 5 shows C as a function of the spread of difficulty. If $\delta \sim N(0, s^2)$, then $C = \mathbb{E}\,\sigma(|\sqrt{2}\,sZ|)$. For our simulated instructor (sd 1.50), $C \approx 0.787$. An instructor whose judgments vary less (a smaller spread) caps every model lower. Section 7 shows how to estimate your own ceiling from repeated pairs.

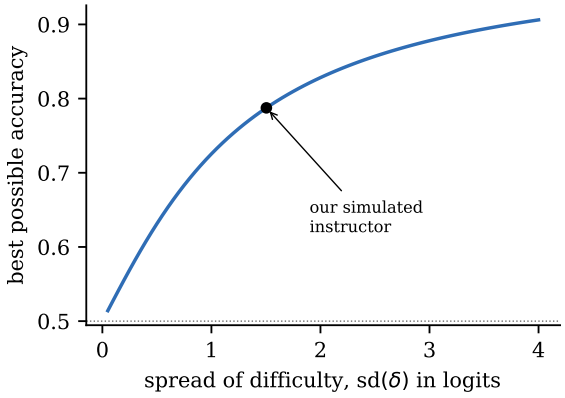


Figure 5: Best achievable pairwise accuracy as a function of how spread out the true difficulties are.

Error bars when comparisons share problems. On the 200 unseen problems, the feature model of Section 6 scores accuracy 0.747 on 4,000 comparisons. The textbook binomial interval

is $\pm 1.96\sqrt{a(1-a)/m} = \pm 0.013$, and it is wrong. The comparisons are not independent: every comparison involving a problem the model misjudges tends to be wrong together. Accuracy is (essentially) a *U-statistic* $U = \binom{n}{2}^{-1} \sum_{i<j} h(x_i, x_j)$ in the problems, and Hoeffding's decomposition gives

$$\operatorname{Var} U = \frac{4\zeta_1}{n} + O(n^{-2}), \qquad \zeta_1 = \operatorname{Var}_x(\mathbb{E}[h(x, x') \mid x]).$$

The precision is governed by the number of *problems* n, not the number of comparisons m. Once $m \gg n$, more comparisons among the same problems barely help. The fix is to resample problems:

From src/calcdiff/evaluate.py

```
for _ in range(n_boot):
    draw = rng.choice(items, size=len(items), replace=True)   # resample PROBLEMS
    c = np.bincount(draw, minlength=n).astype(float)           # multiplicity c_i
    w = c[pairs[:, 0]] * c[pairs[:, 1]]                        # comparison weight c_i c_j
    stats.append(metric(w))
```

If problem i is drawn c_i times, each comparison (i, j) appears $c_i c_j$ times among the resampled problems, and the metric is recomputed with those weights. The result is $[0.716, 0.781]$, a half-width of 0.033, which is $2.4\times$ the naive one. On real data with a few hundred problems, this is the error bar to believe.

Exercises.

Exercise 5.1. Suppose every pair $\{i, j\}$ is labeled once by the true order ($y = \mathbf{1}[\delta_i > \delta_j]$, no ties). Show that the pairwise accuracy of scores s equals $(1 + \tau(s, \delta))/2$.

Exercise 5.2. Show that $q \mapsto \mathbb{E}_{y \sim \mathrm{Bern}(p)}[-y \log q - (1-y)\log(1-q)]$ is uniquely minimized at $q = p$, and that accuracy (as a function of reported q) is not strictly proper.

Exercise 5.3. For the U-statistic above, derive $\operatorname{Var} U = \binom{n}{2}^{-1}(2(n-2)\zeta_1 + \zeta_2)$, where $\zeta_2 = \operatorname{Var} h$. When m comparisons are sampled at random instead of all $\binom{n}{2}$, what extra term appears?

6 Linear Bradley–Terry on features: generalizing to new problems

Now we combine Sections 3 and 4. Difficulty is a linear function of standardized features,

$$\hat{\delta}(x) = \theta \cdot z(x), \qquad \mathbb{P}(x_i \succ x_j) = \sigma(\theta \cdot (z_i - z_j)),$$

so the design matrix has rows $z_i - z_j \in \mathbb{R}^d$ instead of $e_i - e_j \in \mathbb{R}^n$. It is the same convex problem, with the same gradient, Hessian and Newton iteration. `fit_newton` is reused without changing a character. (No intercept: it would cancel in every difference.)

From src/calcdiff/models/feature_bt.py

```
def fit(self, Phi, pairs, y):
    X = self._select(Phi)                        # all features, or a subset for baselines
    seen = np.unique(pairs)                      # problems in TRAINING comparisons only
    self.scaler_ = Standardizer().fit(X[seen])   # no leakage from held-out problems
    Z = self.scaler_.transform(X)
    D = Z[pairs[:, 0]] - Z[pairs[:, 1]]          # rows z_i - z_j
    self.fit_ = fit_newton(D, y, l2=self.l2)
    self.theta_ = self.fit_.theta
    return self

def score(self, Phi):                            # works for ANY problem
    return self.scaler_.transform(self._select(Phi)) @ self.theta_
```

`Phi` holds features for every problem, both labeled and not. `pairs` index into its rows. `score` is where the generalization happens: it never asks whether a problem was ever compared.

Why so few labels suffice: shared information. The Fisher information is now $\sum_k w_k(z_{i_k} - z_{j_k})(z_{i_k} - z_{j_k})^\top$, a $d \times d$ matrix with $d = 17$, instead of the $n \times n$ Laplacian. Every comparison informs *all* problems through θ: a comparison involving one quotient problem teaches the model something about every quotient problem. The variance of a prediction for a brand-new problem x is $z(x)^\top H^{-1} z(x)$, which shrinks like $1/m$ no matter whether x was ever seen. This is the payoff of the restriction we accepted in Section 3. We gave up the ability to tell apart problems with equal features, and in exchange d parameters replace n.

Results. Figure 6(a) trains on m random comparisons among 350 problems and tests on 150 *unseen* ones (8 repetitions per m; bands are ± 1 sd).

- Two ceilings: the oracle (0.776) knows the idiosyncratic part $\tau\varepsilon$ of the planted difficulty, which no feature model can see, while the planted feature part $w \cdot \varphi$ scores 0.754. The gap between them is irreducible *for this feature set*. Only better features can close it.
- With $m = 200$ comparisons (about 1.1 per training problem) the model is within one point of the feature ceiling (0.747). Compare the ≈ 40 comparisons *per problem* item-level BT needed in Section 4.
- Baselines: `size` alone plateaus at 0.711. `deriv_size` alone reaches 0.742, almost as good as all 17 features. In this synthetic world "bigger derivative = harder" is a strong proxy, because the planted weights mostly reward things that also make the derivative bigger. Whether that is true of *your* judgments is an empirical question, and one of the first things we will check with real labels.

Reading the weights. Panel (b) compares learned and planted weights in *raw* units (logits per unit of feature). Since $z_j = (\varphi_j - \mu_j)/s_j$, $\theta \cdot z = \sum_j (\theta_j/s_j)\varphi_j + \text{const}$, so the raw weight is θ_j/s_j (`raw_weights`).

How sure are we of each weight? The Laplace standard error $\sqrt{(H^{-1})_{jj}}/s_j$ gives intervals that contain 13 of the 17 planted weights, fewer than the ≈ 16 that nominal 95% coverage would suggest. Two separate things go wrong, and they are worth telling apart.

- **The standard errors are too small.** Laplace assumes the comparisons are independent. But the planted difficulty has a per-problem part $\tau\varepsilon(x)$, invisible to the features and shared by *every* comparison involving x. This is the dependence of Section 5 again, now hurting the *weights* rather than the accuracy, and the cure is the same: resample problems and refit.

  From `src/calcdiff/models/feature_bt.py`

  ```
  def problem_bootstrap(Phi, pairs, y, l2, rng, n_boot=200):
      for _ in range(n_boot):
          c = np.bincount(rng.choice(items, size=len(items), replace=True), minlength=n)
          idx = np.repeat(np.arange(len(pairs)), c[pairs[:, 0]] * c[pairs[:, 1]])
          out.append(FeatureBT(l2=l2).fit(Phi, pairs[idx], y[idx]).raw_weights())
  ```

  Here the bootstrap standard errors are 1.4× larger (median). Over 6 independent replications with light ridge, Laplace intervals covered 90% of planted weights, and problem-bootstrap intervals 99%, with standard errors 1.9× larger. (An earlier version of these notes reported all 17 covered. That was luck.)
- **Ridge is biased, and a standard error cannot see bias.** Even the bootstrap intervals contain only 14 of 17. The misses are `n_chain`, `n_trig`, `n_exp`, `n_ln`: one family. Every

function applied to something other than x *is* a chain rule, so `n_chain` is nearly a linear combination of the function counts. Faced with near-duplicate features, ridge prefers to *spread* weight across them (spreading lowers $\|\theta\|^2$ for the same predictions). This is ridge's *grouping effect*, and here it moves weight from `n_chain` onto `n_trig`, `n_exp`, `n_ln`. Predictions barely care, since the group's combined effect is what the data pin down. But it is a warning against reading a single weight literally when its feature has near-twins.

Well-determined weights do fine: a quotient rule is worth $0.48 \pm 1.96 \cdot 0.19$ logits (planted 0.55).

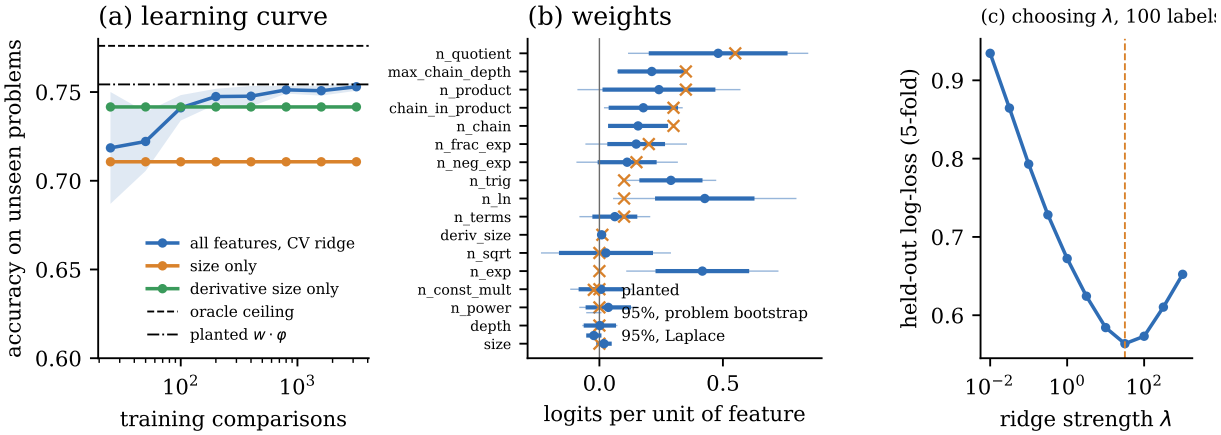


Figure 6: (a) Accuracy on unseen problems vs. number of training comparisons. (b) Learned vs. planted weights. (c) Five-fold cross-validated log-loss against λ with 100 training comparisons.

Choosing λ by cross-validation, and a bug. λ is a *hyperparameter*: it is not learned by minimizing the training loss (which always prefers $\lambda = 0$). It is chosen by estimating the held-out loss. `cv_l2` splits the *problems* into 5 folds. For each fold it trains on comparisons among the other folds' problems and scores the comparisons that touch the held-out fold:

From src/calcdiff/models/feature_bt.py

```
for held in chunks:                          # chunks = 5 disjoint sets of problems
    te = pairs_touching(pairs, held)         # >= 1 held-out problem
    tr = ~te                                 # both problems in the other folds
    if te.sum() == 0 or tr.sum() == 0:
        continue
    for g, l2 in enumerate(grid):
        m = FeatureBT(l2=l2).fit(Phi, pairs[tr], y[tr])
        total[g] += te.sum() * log_loss(m.score(Phi), pairs[te], y[te])
    count += te.sum()
```

The first version required *both* problems of a test comparison to be held out. With 25 comparisons spread over 350 problems, a fold of 70 problems then contained about $25 \cdot (1/5)^2 = 1$ test comparison, sometimes zero. The mean of an empty array is `nan`, and `np.argmin` returns the index of the *first* `nan`. So CV silently "chose" the smallest λ every time, and accuracy with 25 labels looked like 0.615. With the fix it is 0.719. Why is one held-out problem enough? A feature model has no per-problem parameters, so a comparison between a seen problem and an unseen one still requires *scoring an unseen problem from its features*. (For item-level BT this would not be true.)

Panel (c) is the classic U-shaped curve: too little λ overfits, too much underfits. The median chosen λ rises from 20.8 at $m = 25$ to 100 at $m = 3200$, which looks backwards until you notice that

the loss is a *sum* over comparisons. The data term grows like m while the penalty does not, so the effective strength is λ/m, which *falls* from 0.83 to 0.031. In Bayesian terms the prior $\theta \sim N(0, \lambda^{-1}I)$ is fixed, and it matters less as data accumulate.

Exercises.

Exercise 6.1. Show that minimizing $\frac{1}{m}\sum_k \ell_k + \frac{\lambda'}{2}\|\theta\|^2$ is equivalent to our objective with $\lambda = m\lambda'$. Which parametrization would you expect CV to make roughly constant in m, and why?

Exercise 6.2. Let H be the Hessian at $\hat{\theta}$. Show the Laplace predictive variance of $\hat{\delta}(x) - \hat{\delta}(x')$ is $(z - z')^\top H^{-1}(z - z')$. Why does it not depend on whether x or x' appeared in training?

Exercise 6.3. Explain precisely why "touching" test comparisons are a valid generalization test for the feature model but not for item-level BT. Is the standardizer a (small) exception? (Look at which problems `fit` uses for μ, s.)

7 Collecting real labels

Everything so far used a simulated instructor. This section builds the tool for the real one, and the theory for interpreting what it collects. Run it with

```
.venv/Scripts/python -m streamlit run labeler/app.py
```

and open `http://localhost:8501`. It shows two problems side by side. You click *Left is harder*, *About the same*, or *Right is harder*, and a new pair appears. A sidebar shows your count and your self-agreement on repeats, and has an *Undo* button. Under each problem there is a *Flag as unnatural* button that removes the problem from rotation.

What is recorded, and why. Every click is one row in `data/labels.sqlite`:

From `src/calcdiff/labels/store.py`

```
CREATE TABLE judgments (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    left_id     TEXT NOT NULL REFERENCES problems(id),
    right_id    TEXT NOT NULL REFERENCES problems(id),
    outcome     TEXT NOT NULL CHECK (outcome IN ('left', 'right', 'tie')),
    labeler     TEXT NOT NULL,
    response_ms INTEGER,       -- time from showing the pair to the click
    is_repeat   INTEGER NOT NULL DEFAULT 0,
    created_at  REAL NOT NULL
);
```

Each column is there for a statistical reason:

- **Screen position** (`left_id`, `right_id`, and the outcome *relative to the screen*). People prefer one side more often than chance. With position recorded, we can fit $\mathbb{P}(\text{left} \succ \text{right}) = \sigma(\delta_L - \delta_R + \beta)$. In design-matrix terms, β is an *intercept*, a column of ones in D. Section 6 said an intercept cancels in differences. That is true for difficulty, but not for a bias tied to the *order* of presentation. Randomizing which problem goes left makes β statistically separable from δ.
- **Repeats** (`is_repeat`). About 10% of pairs are earlier pairs shown again with the sides swapped (so you can't just remember a position). They measure your own noise (below).

- **Response time**. Slow judgments are plausibly close calls. We may later use it as a soft signal of $|\delta_i - \delta_j|$.
- **Flags** (a separate table). These are data about the *generator* rather than about difficulty: they show which corners of the grammar (Section 1) produce problems nobody would assign.

From clicks to training labels, including ties. `to_arrays` turns rows into exactly what the models of Sections 4 and 6 consume: pairs (L, R) and $y = 1, 0, \frac{1}{2}$ for left/right/tie. A tie with $y = \frac{1}{2}$ contributes $\log(1 + e^t) - \frac{t}{2}$ to the cross-entropy, which is exactly half a win plus half a loss (a test checks this). That is the simplest principled treatment. Davidson's model adds a separate tie parameter ν, $\mathbb{P}(\text{tie}) \propto \nu\sqrt{\pi_i \pi_j}$, which we can try if you use the middle button a lot.

The scheduler. **From** src/calcdiff/labels/schedule.py

```python
def next_pair(store, labeler, rng):
    flagged = store.flagged()
    ids = [p.id for p in store.problems() if p.id not in flagged]
    past = [j for j in store.judgments(labeler) if not j.is_repeat and ...]
    if len(past) >= MIN_BEFORE_REPEATS and rng.random() < P_REPEAT:
        j = past[rng.integers(len(past))]
        return Pair(j.right_id, j.left_id, True)    # same pair, sides swapped
    i, k = rng.choice(len(ids), size=2, replace=False)
    return Pair(ids[i], ids[k], False)
```

For now pairs are uniformly random among the 300 pool problems. Section 4 told us this wastes effort: lopsided pairs have conductance $\sigma'(\Delta) \approx 0$. Section 10 replaces the last two lines with active learning, which picks the pair whose answer would teach the model the most.

How the app works (Streamlit in one paragraph). Streamlit re-executes `labeler/app.py` from top to bottom on *every* interaction. State that must survive a rerun lives in `st.session_state` (per browser tab: the current pair and the time it was shown) or in the database (permanent). The database connection is created once per server process by a function decorated with `@st.cache_resource`. On the very first launch, that function also generates the pool of 300 problems (`Sampler(seed=2027)`). Buttons call functions *before* the rerun via `on_click`:

From labeler/app.py

```python
def submit(outcome: str) -> None:
    ms = int(1000 * (time.time() - state.shown_at))
    store.record(pair.left_id, pair.right_id, outcome, labeler, ms, pair.is_repeat)
    new_pair()                                  # choose the next pair; the rerun shows it

b1.button("Left is harder", on_click=submit, args=("left",), type="primary")
```

Thurstone or Bradley–Terry? Why the logistic link? Thurstone (1927) modeled a judgment as comparing *noisy perceptions* $\delta_i + \varepsilon_i$ and $\delta_j + \varepsilon_j$. With Gaussian ε this gives $\mathbb{P}(i \succ j) = \Phi((\delta_i - \delta_j)/\sigma\sqrt{2})$, the *probit* model. With i.i.d. *Gumbel* ε, the difference is logistic, and we get exactly Bradley–Terry (Exercise 7.1). Which noise is right? The data can barely tell: $\Phi(t/1.702)$ differs from $\sigma(t)$ by at most 0.0095 anywhere (Figure 7(a)). We choose the logistic for convenience: closed-form gradients, and the same mathematics as logistic regression.

Reading your own consistency. Suppose that on a pair with gap Δ you pick the harder problem with probability $p = \sigma(|\Delta|)$, independently each time it is shown. Then on a repeated pair you

agree with yourself with probability $p^2 + (1-p)^2$, while the best any model can do on that pair is p. Averaging over pairs gives a curve linking two quantities: your overall self-agreement A, which the app measures, and the ceiling C on accuracy against your labels (Figure 7(b)). For example, self-agreement of 70%, 80% or 90% means no model can exceed about 0.780, 0.858 or 0.930 accuracy on your labels. This is the number to compare our models against once real labels arrive: a model at 0.80 is excellent if your ceiling is 0.82, and mediocre if it is 0.93.

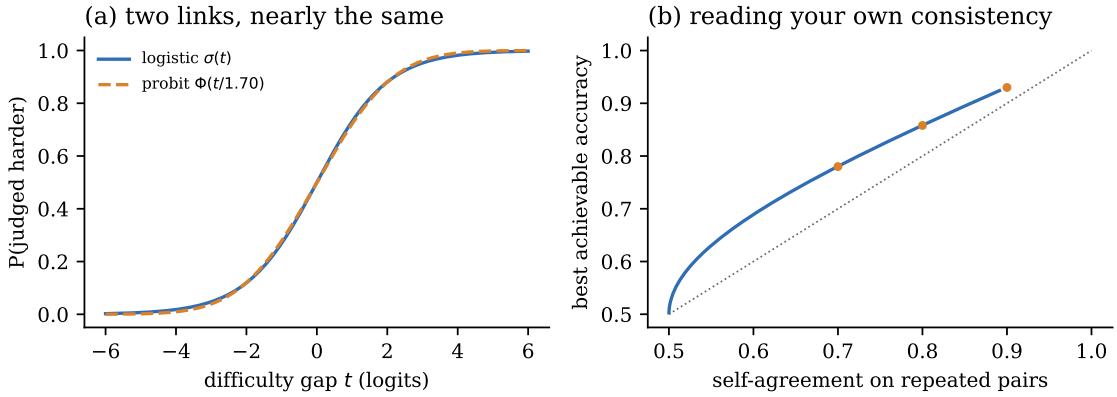


Figure 7: (a) Logistic and rescaled probit links. (b) Self-agreement on repeated pairs vs. the best accuracy achievable against those labels, under the model of the last paragraph. Dots mark 70%, 80%, 90% agreement.

A labeling protocol. Some suggestions for when you sit down to label:

- Decide what "harder" means, write it down, and stick to it. The app's prompt is "for a typical Calculus I student, the work to find and write down the derivative".
- Go fast. Your first instinct is the measurement, and agonizing adds noise, not signal. Use *About the same* when it really is.
- Flag freely. A flagged problem costs nothing, while a weird problem in the training data costs accuracy.
- 150 comparisons (about 20 minutes) is enough for a first model (Section 8). Section 6 says the feature model is near its ceiling at roughly one comparison per problem.

Exercises.

Exercise 7.1. Let $\varepsilon_i, \varepsilon_j$ be i.i.d. standard Gumbel, with $\mathbb{P}(\varepsilon \le u) = e^{-e^{-u}}$. Show that $\varepsilon_i - \varepsilon_j$ is standard logistic, so that $\mathbb{P}(\delta_i + \varepsilon_i > \delta_j + \varepsilon_j) = \sigma(\delta_i - \delta_j)$.

Exercise 7.2. Writing the per-pair agreement as $a = p^2 + (1-p)^2$ with $p \ge \frac{1}{2}$, show $p = \frac{1}{2}(1+\sqrt{2a-1})$, and use Jensen's inequality to prove the distribution-free bound $C \le \frac{1}{2}(1+\sqrt{2A-1})$. How close is the curve in Figure 7(b) to this bound?

Exercise 7.3. With position randomized uniformly, show that the MLE of β in $\sigma(\delta_L - \delta_R + \beta)$ is asymptotically uncorrelated with the δ's. What goes wrong if the harder problem is shown on the left more often?

8 Real labels: what the model learned about you

Everything up to now was rehearsal on a simulated instructor. This section fits the same code to 330 judgments you made in the labeling app, and asks the questions Section 5 prepared: how good is the model, compared with what, and how sure are we?

The data. The 330 judgments include 30 repeats and 56 ties (17%) and cover 249 of the 300 pool problems, about two comparisons per problem. The median time per judgment was 1.9 s. Among decided comparisons you chose the left problem 0.504 of the time, so there is no detectable position bias (Section 7), and the intercept β is not worth fitting. You flagged 0 problems. The code path from the database to the model is the one from Section 7: `store.to_arrays("dylan")` gives `(problems, pairs, y)`, and `feature_matrix` gives Φ. Every number in this section is recomputed from the live database whenever the notes are rebuilt, so it reflects the judgments as of the last build.

Your ceiling, and a subtlety about ties. You agreed with yourself on 25/30 repeats. Read naively through Figure 7(b), that gives a ceiling around 0.87, and yet the models below score 0.914 and 0.925. Something had to give. Of your 5 disagreements, 3 involve a *tie* ("about the same" one time, a decision the other). Accuracy skips tied labels, so the right comparison is agreement when *both* showings were decided: 20/22. With so few repeats the uncertainty is large. The Jeffreys interval (the posterior quantiles under a Beta$(\frac{1}{2}, \frac{1}{2})$ prior, which has good frequentist coverage for small counts) is $[0.74, 0.98]$, which maps to a ceiling of 0.94 with range $[0.81, 0.99]$. So the models are *at* your ceiling, as far as 22 repeats can tell. More repeats would sharpen this, and they are cheap: they come up automatically.

Cross-validated comparison. Every number below is out-of-fold: 5-fold CV over *problems*, repeated 5 times. Inside each training fold, λ is chosen by a second, inner CV (*nested* CV), so the held-out problems never influence any choice.

From src/calcdiff/evaluate.py

```
def cross_val_diffs(Phi, pairs, y, make_model, rng, folds=5):
    ...
    for held in chunks:                          # disjoint sets of problems
        te = pairs_touching(pairs, held)
        model = make_model(pairs[~te], y[~te])   # never sees 'held'; may run its own CV
        s = model.score(Phi)
        total[te] += s[pairs[te, 0]] - s[pairs[te, 1]]
        count[te] += 1
    return total / count                         # a comparison spanning 2 folds: average
```

Passing a *factory* `make_model` rather than a model is what makes nesting easy: the factory for the full model calls `cv_l2` on whatever training comparisons it is handed.

Table 5 has two surprises.

- **One feature does almost everything.** `deriv_size` alone orders 0.925 of your decided comparisons correctly (problem-bootstrap 95% interval $[0.868, 0.972]$). The full model gets 0.914, with interval $[0.843, 0.961]$. As a working hypothesis about how you judge, *harder means more to write down* is very good.
- **Accuracy and log-loss disagree.** By log-loss the full model is better: 0.310 vs. 0.378. Is that real? Comparing the two separate intervals is the wrong test: they overlap, but the two models are scored on the *same* comparisons, so their errors are correlated. A *paired* bootstrap

model	accuracy	log-loss
constant (coin flip)	0.500	0.693
`size` only	0.861	0.434
`n_chain` only	0.850	0.403
`depth` only	0.870	0.394
`deriv_size` only	0.925	0.378
all 17 features, CV ridge	0.914	0.310

Table 5: Out-of-fold accuracy and log-loss on your labels. Single-feature models are one-parameter Bradley–Terry fits.

resamples problems once and recomputes *both* losses on that resample. For the difference (full minus single-feature) it gives $[-0.183, 0.000]$. Nearly all of the mass is below zero, with the upper end at about zero, so the advantage is likely but not yet certain. (With 319 judgments the interval sat just below zero. Eleven more nudged it to the boundary, a reminder of how soft a "significant" result near the edge is.) Accuracy only checks the sign of $\hat{\delta}_i - \hat{\delta}_j$, while log-loss also checks its size. The full model knows *how much* harder one problem is than another: when it is confident it is nearly always right, and when it is unsure it says so. That calibration matters downstream: comparing raters (Section 13) and an adaptive test (Section 14) both use the probabilities, not just the order.

Learning curves and the bias–variance trade-off. Why does the 1-feature model win on accuracy while the 17-feature model wins on log-loss? Fitting d parameters to m comparisons has a predictable cost. For a correctly specified model fit by maximum likelihood, a second-order Taylor expansion around the true θ^* (Exercise 8.1) gives

$$\mathbb{E}[\text{held-out loss}] - \mathbb{E}[\text{training loss}] \;\approx\; \frac{d}{m} \qquad \text{(Akaike, 1973).}$$

The training loss underestimates the truth by $\frac{d}{2m}$, because $\hat{\theta}$ was fit to these very points. The held-out loss overestimates it by another $\frac{d}{2m}$, because $\hat{\theta} \neq \theta^*$. With a ridge penalty, d is replaced by the *effective degrees of freedom*

$$\mathrm{df}(\lambda) = \mathrm{tr}\big(H_{\text{data}}(H_{\text{data}} + \lambda I)^{-1}\big) = \sum_i \frac{\mu_i}{\mu_i + \lambda},$$

where μ_i are the eigenvalues of the data Hessian. Each eigen-direction counts fully if the data dominate it ($\mu_i \gg \lambda$) and not at all if the prior does. That's the Section 3 picture again: collinear directions have small μ_i, and ridge switches them off.

From `scripts/figs_step08.py`

```
_, H_data = grad_hess(mdl.theta_, D, y[sub], 0.0)          # Hessian of the data term only
df = np.trace(H_data @ np.linalg.inv(H_data + mdl.l2 * np.eye(len(H_data))))
```

Figure 8(a) holds out a fold of problems and trains on m random comparisons from the rest (20 repetitions per m). The full model's dotted line, training loss $+\,\mathrm{df}/m$, tracks its held-out loss: predicted gap 0.229 vs. observed 0.148 at $m = 25$, and 0.054 vs. 0.053 at $m = 200$ (where df ≈ 10.8 of 17). The prediction runs high at small m, which is plausible: λ is re-tuned at each m, the theory assumes a fixed model, and it is an $m \to \infty$ expansion. The $1/m$ shape is right. This is the bias–variance trade-off in its cleanest form:

- `deriv_size` only: df = 1, so almost no variance. But it plateaus at the log-loss of the best *one-feature* model. That plateau is its *bias*.
- All features: more variance (a df/m penalty), less bias. With 25 comparisons it already beats the simple model on log-loss (0.381 vs. 0.484), but not on accuracy (0.903 vs. 0.925).

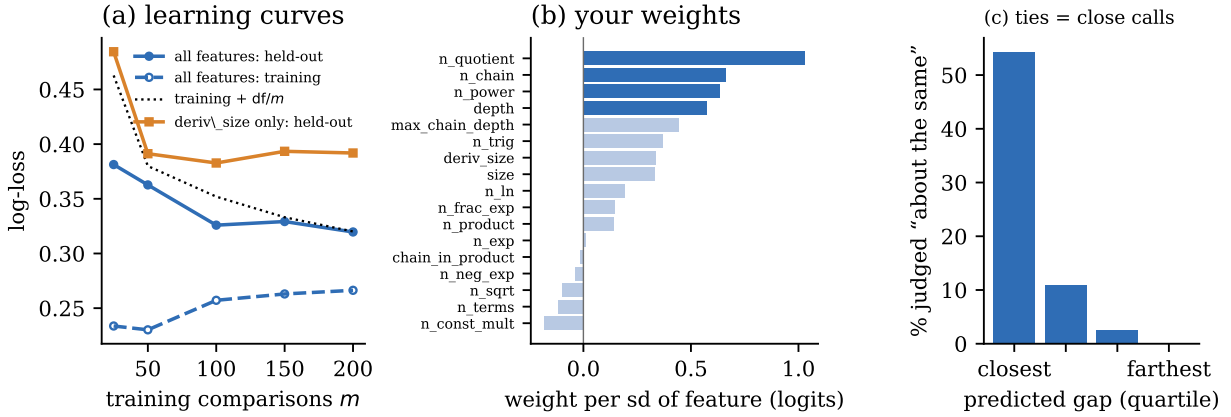


Figure 8: Your labels. (a) Training and held-out log-loss vs. training-set size, and the Akaike/TIC prediction. (b) Standardized weights of the final model (dark: problem-bootstrap $|z| > 1.96$). (c) Fraction of "about the same" answers by quartile of the model's predicted gap.

What the weights say about you. Fit on all comparisons (CV chooses $\lambda = 3.16228$), the final model's scores have Spearman correlation 0.97 with derivative size. Intervals here come from the problem bootstrap of Section 6, not from Laplace. (On your labels the two happen to agree, with a median ratio of 1.0. That suggests the features leave little problem-specific residual in your judgments, unlike the simulated instructor, whose $\tau\varepsilon$ the features could not see.) Only 4 weights have bootstrap $|z| > 1.96$. That is the collinearity of Section 3 at work: the data can't split credit among `size`, `depth`, the rule counts and `deriv_size`, which all move together. Panel (b) shows the standardized weights. The largest are on `n_quotient`, `n_chain`, `n_power`, `depth`.

One effect stands clear of the rest, and it shows how to read a weight carefully. **A quotient rule adds** 2.9 **logits** beyond everything else the model sees: bootstrap 95% interval $[1.2, 4.5]$, positive in 100.0% of 500 resamples. The *direction* is solid. You treat quotients as harder than their size alone explains, which is a genuinely pedagogical finding, and the kind of thing to check on a second instructor. The *size* is not solid. Only 13% of the problems you saw contain a quotient, so the estimate leans on few comparisons and on the prior. It is 4.6 logits at $\lambda = 0.316$ and 1.3 at $\lambda = 31.6$, against 2.9 at the CV choice. A weight that moves this much with the regularization should be reported as a direction with a wide range, not as "a quotient multiplies the odds by $e^{2.9}$".

Do ties mean "close"? A check on the model itself. Bradley–Terry never saw a tie as anything but half a win. If the model is a good description of your judgments, your ties should concentrate where its *out-of-fold* predicted gap $|\hat{\delta}_i - \hat{\delta}_j|$ is small. Figure 8(c) shows exactly that: 54% ties in the closest quartile and 0% in the farthest. Your response times agree: a median of 2.4 s on the closest pairs vs. 1.3 s on the most lopsided. A model that predicts things it was never trained on (ties, hesitation) is a model worth trusting.

Error analysis. The model gets 25 decided comparisons wrong out-of-fold. Table 6 lists the five it was most confident about. Reading your own disagreements with a model is the most useful thing in this section. Some are probably slips: with a 1.9-second median, a few misclicks are inevitable, and the most confident miss is a candidate. Others are real disagreements about what makes a problem hard. For instance, you rated $8\sin(\ln(x^2)+7)$ harder than $\cos(\cos(e^{x^3}-5))$, while the model, which counts nested compositions, disagreed. Where you think *you* are right, the remedy is a feature, not more data.

you said harder	model said harder	P_{model}
$x^2+\left(\left(x^{1/3}+3\right)^4-3\right)^2$	$x^3+7\left(-4x^{1/2}\right)\sin\left(x^{-1}e^{-5x^2+1}\right)$	0.98
$x^4+x^3+\sqrt{\sin\left(x^3\right)+5}+4$	$x^{-1}+x^{1/2}+\dfrac{\sqrt{4x^3}}{2x^{3/2}}+2$	0.94
$x^{-1/2}e^{x^4}+4$	$\ln x+\sqrt{x}+e^{-7x^3}\cos\left(x^{3/2}-8\right)$	0.91
$8\sin\left(\ln\left(x^2\right)+7\right)$	$\cos\left(\cos\left(e^{x^3}-5\right)\right)$	0.84
$\dfrac{e^{x^{3/2}}}{\left(x^2-3\right)^2}+4x+2\left(7x^3+1\right)^5$	$-2x^2+2\left(\sqrt{e^{x^2+2}}-6\right)^4\cdot 9x\sin\left(x^{-2}\right)x$	0.81

Table 6: The model's five most confident out-of-fold mistakes. P_{model} is the probability it gave to the answer you did not choose.

The table also caught a bug in Section 1's generator. Terms like $3\cdot 9x(\ldots)$ and $7(-4x^{1/2})\sin(\ldots)$ stack two coefficients, which `is_natural` should have rejected and no instructor would write. (You didn't flag them, perhaps because they read as merely ugly rather than wrong.) The fix is in Section 1. Under the tightened rule, 12 of the 300 pool problems are unnatural, and 26 of your judgments involve one. What to do with data collected under a bug? Here the judgments are still honest comparisons of real expressions, so they stay in the training set. The 12 problems are flagged under a separate labeler name, `auto:is_natural`, which takes them out of rotation without being mistaken for your own flags. Then a *robustness check*: refit without the 26 judgments. Accuracy is 0.923 and log-loss 0.299 (vs. 0.914 and 0.310), and the quotient weight is 2.9 with interval $[1.2, 4.6]$. The conclusions do not hinge on the bad problems. Data inspection found this bug, not a test, and that is typical.

Exercises.

Exercise 8.1. Let ℓ be a log-loss with $\theta^* = \arg\min \mathbb{E}\ell$, $J = \mathbb{E}\nabla^2\ell(\theta^*)$, $K = \operatorname{Var}\nabla\ell(\theta^*)$. Show that, to order $1/m$, $\mathbb{E}[\text{held-out}] - \mathbb{E}[\text{train}] = \operatorname{tr}(J^{-1}K)/m$ (Takeuchi), and that $J = K$ for a correctly specified likelihood, giving d/m.

Exercise 8.2. Show that $\operatorname{df}(\lambda) = \sum_i \mu_i/(\mu_i+\lambda)$ is decreasing in λ, with $\operatorname{df}(0) = \operatorname{rank} H_{\text{data}}$. Interpret each term as the fraction of the i-th eigen-direction that is "learned from data" rather than "taken from the prior".

Exercise 8.3. Construct three problems and probabilities for which a model A orders every pair correctly but has worse expected log-loss than a model B that misorders one pair. What does this say about which metric to use for model *selection* vs. *reporting*?

9 Gradient boosting: gradient descent in function space

The linear model can only add up feature effects: one more chain rule is worth the same everywhere. Section 3 handled *one* interaction by hand (`chain_in_product`) and noted that this does not scale. Gradient-boosted trees learn nonlinearities and interactions automatically. They are the default winner on tabular data in practice, and the idea behind them is a beautiful piece of optimization.

Descent on scores, projected onto trees. Forget parameters for a moment, and optimize the scores $F_i = \hat{\delta}(x_i)$ of the training problems directly. The loss $L(F) = \sum_k [\log(1+e^{F_i - F_j}) - y_k(F_i - F_j)]$ is the Bradley–Terry loss with one-hot design, so (Section 4) its gradient and Hessian are

$$g = \nabla_F L = D^\top(p - y), \qquad \nabla_F^2 L = D^\top W D = L_w, \text{ the weighted Laplacian.}$$

Plain gradient descent on F would just fit item-level BT. It moves each F_i independently and says nothing about unseen problems. Boosting instead *restricts every step to a function of the features*. Each round fits a small regression tree $f_t : \mathbb{R}^d \to \mathbb{R}$ to the descent direction, then steps $F \leftarrow F + \nu f_t(\varphi(x))$. After T rounds, $\hat{\delta}(x) = \nu \sum_t f_t(\varphi(x))$ can score any problem. Friedman (2001) called this *gradient boosting*.

Which tree? Expand L to second order along a step f, replacing L_w by its diagonal $h_i = (L_w)_{ii}$:

$$L(F+f) \approx L(F) + \sum_i g_i f_i + \tfrac{1}{2}\sum_i h_i f_i^2 = \tfrac{1}{2}\sum_i h_i\Big(f_i + \frac{g_i}{h_i}\Big)^2 + \text{const.}$$

So the best tree is a *weighted least-squares* fit to the per-problem Newton steps $-g_i/h_i$, with weights h_i. That is a standard regression tree, exactly what `DecisionTreeRegressor` fits.

From src/calcdiff/models/boosted_bt.py

```
def problem_grad_hess(F, pairs, y):
    p = expit(F[pairs[:, 0]] - F[pairs[:, 1]])
    r = p - y                       # d loss_k / d t_k: the "lambda" of LambdaRank
    w = p * (1 - p)                 # d2 loss_k / d t_k^2
    g = np.bincount(pairs[:, 0], r, n) - np.bincount(pairs[:, 1], r, n)
    h = np.bincount(pairs[:, 0], w, n) + np.bincount(pairs[:, 1], w, n)
    return g, h, w
```

`np.bincount(idx, weights, n)` sums `weights` into bins `idx`: it is $D^\top r$ without forming D. (A test checks that g and h agree with `grad_hess` on the one-hot design.) The quantity $r_k = p_k - y_k$ is what the ranking literature calls the *lambda* of a pair, and *LambdaMART*, the algorithm behind many search engines' rankers, is precisely this construction.

The leaf values: an exact Newton step, and a graph cut. The tree decides the *partition*. We then choose each leaf's value by an exact one-dimensional Newton step instead of the diagonal approximation. Shifting the scores of all problems in a leaf R by γ moves F along $\mathbf{1}_R$, so

$$\frac{d}{d\gamma} L(F + \gamma \mathbf{1}_R) = \mathbf{1}_R^\top g = \sum_{i \in R} g_i,$$
$$\frac{d^2}{d\gamma^2} L(F + \gamma \mathbf{1}_R) = \mathbf{1}_R^\top L_w \mathbf{1}_R = \sum_k w_k\big(\mathbf{1}_R(i_k) - \mathbf{1}_R(j_k)\big)^2 = \mathrm{cut}_w(R).$$

The curvature in direction $\mathbf{1}_R$ is the total weight of comparisons with exactly one problem in R. A comparison *inside* the leaf is unaffected by shifting both of its problems together. That is the

Laplacian quadratic form from Section 4 again, now evaluated on an indicator vector, where it becomes a graph cut. The diagonal approximation would use $\sum_{i \in R} h_i = \mathrm{cut}_w(R) + 2w(\text{inside } R)$ (Exercise 9.1), which overestimates the curvature and makes the steps too timid.

From src/calcdiff/models/boosted_bt.py

```
for _ in range(self.n_trees):
    g, h, w = problem_grad_hess(F, pairs, y)
    tree.fit(Phi[items], -g[items] / h[items], sample_weight=h[items])  # partition
    leaf_of = tree.apply(Phi)                                  # leaf id of EVERY problem
    G = np.bincount(leaf_of[items], g[items], n_nodes)         # sum of g per leaf
    value = -G / (leaf_cut(leaf_of, pairs, w, n_nodes) + self.leaf_l2)  # Newton
    F += self.learning_rate * value[leaf_of]                   # shrunken step

def leaf_cut(leaf_of, pairs, w, n_leaves):
    li, lj = leaf_of[pairs[:, 0]], leaf_of[pairs[:, 1]]
    cross = li != lj                                           # comparisons crossing a leaf
    return (np.bincount(li[cross], w[cross], n_leaves)
            + np.bincount(lj[cross], w[cross], n_leaves))
```

Two tests pin the mathematics down. `test_leaf_curvature_is_the_laplacian_cut` checks $\mathrm{cut}_w(R) = \mathbf{1}_R^\top L_w \mathbf{1}_R$ against the explicit Hessian, and `test_leaf_value_is_exact_newton_step_for_one_leaf` checks γ against finite differences. `leaf_l2` adds a ridge penalty $\frac{\lambda}{2}\gamma^2$ to each leaf (the XGBoost convention).

What trees can express, and what they ignore. A tree of depth D is a piecewise-constant function on axis-aligned boxes, and each leaf is reached by at most D comparisons $\varphi_j \le c$. Hence:

- **Depth controls the order of interactions.** A depth-1 tree (a "stump") splits on one feature, so a sum of stumps is an *additive* model $\sum_j f_j(\varphi_j)$: nonlinear in each feature, but with no interactions. Depth D allows interactions among up to D features.
- **Invariance to monotone transformations.** Replacing φ_j by any strictly increasing $u(\varphi_j)$ yields the same partitions, hence the same model (`test_invariant_to_monotone_feature_transforms`). No standardization is needed. Contrast this with ridge (Section 3).
- **Regularization** comes from four knobs: the shrinkage ν, the depth, the minimum leaf size, and above all the *number of trees*, chosen by early stopping. `cv_n_trees` fits once per fold and reads off the held-out loss after every tree, so the whole curve costs a single fit.

Inductive bias decides the winner. Figure 9(a,b) compares linear BT with boosted trees (depth 2) on two synthetic truths, using held-out log-loss.

- **Linear truth** (the simulated instructor of Section 4): linear wins, by a lot at small m (0.552 vs. 0.580 with 100 comparisons) and still at $m = 3200$ (0.537 vs. 0.541). Trees approximate a linear function with a staircase, which wastes data.
- **Interaction truth**: chain rules make a problem *harder* when a quotient is present and *easier* without one. No linear model can express that. Boosting overtakes linear from $m = 400$ and approaches the oracle (0.515 vs. 0.511, linear 0.520).

Neither model is "better". Each encodes a bet about the truth, and data decide which bet pays (the *no-free-lunch* theorem made concrete). Note the sizes of the gaps, too. Even a sign-flipping interaction shows up only modestly in the averages, because quotients are rare. Interactions that touch few problems are hard to detect, for any method.

On your labels. Out-of-fold over 330 judgments (5-fold CV over problems, repeated 5 times; the number of trees is chosen by inner CV), boosting scores accuracy 0.938 and log-loss 0.290, against

0.909 and 0.304 for linear BT. The paired problem-bootstrap interval for the log-loss difference (boosted minus linear) is $[-0.065, 0.024]$, which straddles zero. *At this sample size the two are indistinguishable.* That itself is information: whatever structure trees could add, 330 comparisons do not reveal it.

Figure 9(c) asks the sharper question of which tree depth is best. Held-out log-loss bottoms out at 0.303 for depth 1 (after 54 trees), 0.312 for depth 2, and 0.316 for depth 4. Then every curve turns upward as the trees start fitting noise. Depth 4 ends at 0.355 held-out while its training loss falls to 0.150, the textbook signature of overfitting. **The best model is a sum of stumps: an additive model with no interactions.** So there is no evidence of interactions in how you judge difficulty, at least none these features can express. Whatever boosting gains over linear BT comes from bending individual features, not from combining them. The boosted model leans mostly on `deriv_size`(43%), `size`(20%), `depth`(12%) (share of total split gain). As in Section 8, that is "how much there is to write" doing most of the work.

A caution about gain: it credits whichever of several correlated features a tree happened to split on, so it is not a reliable account of *why* the model predicts what it does. Section 12 replaces it with Shapley values, which have axioms behind them.

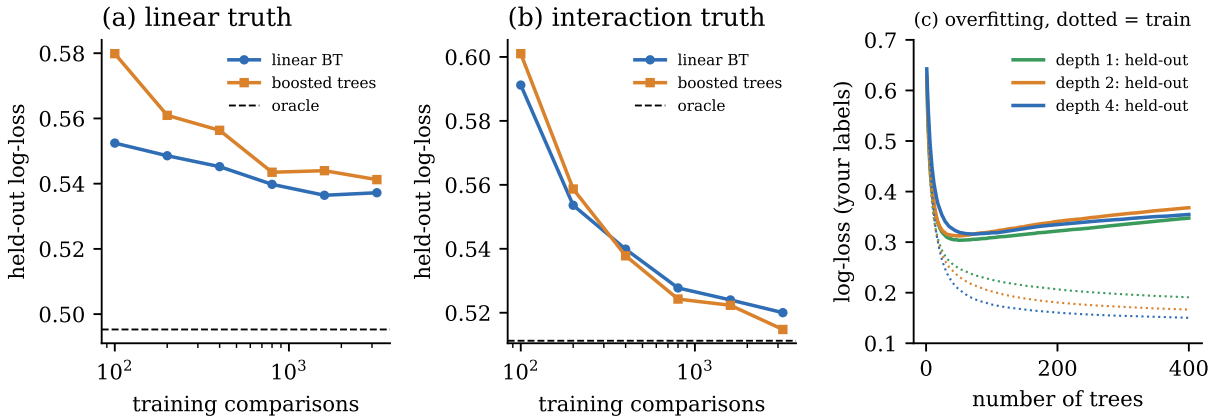


Figure 9: (a,b) Held-out log-loss of linear and boosted Bradley–Terry on synthetic truths, against training-set size. (c) Your labels: held-out (solid) and training (dotted) log-loss against the number of trees for depths 1, 2 and 4.

Exercises.

Exercise 9.1. Show that $\sum_{i \in R} (L_w)_{ii} = \mathrm{cut}_w(R) + 2\, w(E(R))$, where $E(R)$ is the set of comparisons with both problems in R. Conclude that the diagonal Newton step $-\sum_{i \in R} g_i / \sum_{i \in R} h_i$ is never larger in magnitude than the exact one.

Exercise 9.2. Prove that boosting with depth-1 trees produces a function of the form $\sum_j f_j(\varphi_j)$ with each f_j a step function, and that every such function on a finite data set can be reached. Which simple feature-wise shape (monotone? convex?) would you impose on $f_{\texttt{deriv_size}}$ as an instructor, and how could you enforce it?

Exercise 9.3. Show that boosting with a learning rate $\nu \to 0$ and $T\nu$ fixed traces a path that, for an *additive* model class, resembles the ridge path in λ: early stopping acts as regularization. (A heuristic argument suffices. Look up "boosting as ℓ_1-regularized path following" for the precise statement in the linear case.)

10 Active learning: which question to ask next

Your time is the scarce resource. Section 4 showed that comparisons are not equally informative: a lopsided pair has conductance $\sigma'(\Delta) \approx 0$ and teaches almost nothing. So instead of showing random pairs, let the current model choose. This section is a cautionary tale with a happy ending. The obvious choice rules made things *worse*, and finding out why led to a better model, not just a better rule.

Bayesian experimental design. After fitting, the model has an approximate posterior $\theta \sim N(\hat{\theta}, \Sigma)$, with $\Sigma = H^{-1}$ (Laplace, Section 4). A candidate pair with feature difference $d = z_i - z_j$ has predicted probability $p = \sigma(\hat{\theta} \cdot d)$, and asking it would add $w\, dd^\top$ to the precision, with $w = p(1-p)$. Notably, the update does not depend on the *answer*, so we can evaluate it before asking. Two consequences follow from rank-one linear algebra:

- *Matrix determinant lemma*: $\det(H + w\, dd^\top) = \det H\,(1 + w\, d^\top \Sigma d)$. The entropy of a Gaussian is $\frac{1}{2} \log\det \Sigma + \text{const}$, so the expected information gain about θ is

$$\text{EIG} = \tfrac{1}{2} \log(1 + w\, s^2), \qquad s^2 = d^\top \Sigma d.$$

  Maximizing it greedily is sequential *D-optimal* design ("D" for determinant).
- *Sherman–Morrison*: $\Sigma' = \Sigma - c\,\Sigma dd^\top \Sigma$ with $c = w/(1 + ws^2)$. If what we care about is predicting *typical* pairs, i.e. the average predictive variance $\mathbb{E}_{d'}[d'^\top \Sigma d'] = \operatorname{tr}(\Sigma C)$ with $C = \mathbb{E}[d'd'^\top]$ over random pairs, then the pair that reduces it most maximizes

$$c\, d^\top \Sigma C \Sigma d.$$

  That is *V-optimal* design ("V" for variance; Kiefer's alphabet has many more letters).

The simplest heuristic, *uncertainty sampling*, uses w alone: it asks the closest call. Both w and s^2 matter. A close call (w large) whose answer moves the model in a direction it is already sure about (s^2 small) is wasted.

From src/calcdiff/active.py

```
Z = model.scaler_.transform(model._select(Phi))
D = Z[cand[:, 0]] - Z[cand[:, 1]]                  # one row d_k per candidate pair
p = expit(D @ model.theta_)
w = p * (1 - p)
if strategy == "uncertainty":
    return w
Sigma = model.fit_.covariance()
s2 = np.einsum("kj,jl,kl->k", D, Sigma, D)         # d_k^T Sigma d_k, all k at once
if strategy == "information":
    return 0.5 * np.log1p(w * s2)                   # D-optimal
if strategy == "variance":
    C = 2 * np.cov(Z[np.unique(cand)], rowvar=False)   # E[d d^T] for random pairs
    SD = D @ Sigma
    return w / (1 + w * s2) * np.einsum("kj,jl,kl->k", SD, C, SD)   # V-optimal
```

`np.einsum("kj,jl,kl->k", D, Sigma, D)` computes all ≈45,000 quadratic forms $d_k^\top \Sigma d_k$ in one vectorized call, with no Python loop. (Why $C = 2\,\text{Cov}(z)$? If z, z' are independent draws, $\mathbb{E}[(z - z')(z - z')^\top] = 2\,\text{Cov}(z)$.) The test `test_information_gain_is_half_log_det_ratio` checks the EIG formula against explicit log-determinants. The function `choose` then takes the argmax, except with probability 10% it picks uniformly at random (*exploration*). A purely greedy learner can lock onto whatever its current, possibly wrong, model finds interesting and never see evidence against it.

The simulation loop. From src/calcdiff/active.py

```
def ask(k):                                          # each pair is answered exactly once
    chosen.append(k); labels.append(answer(k)); remaining[k] = False

for k in rng.choice(len(cand), size=n_start, replace=False):
    ask(int(k))                                      # a random start
while True:
    model = make_model().fit(Phi, cand[chosen], np.array(labels))
    ... evaluate(model) ...
    for _ in range(batch):                           # 'batch' picks per refit
        left = np.flatnonzero(remaining)
        ask(int(left[choose(model, Phi, cand[left], strategy, rng, explore)]))
```

`answer(k)` is either a simulated instructor or a lookup of one of your stored judgments, so the same loop runs both experiments below. My first version called `answer` again at every refit. For a simulated instructor that re-draws the coin and gives a pair a *different* label each time. The `ask` helper fixes this, and `test_loop_asks_each_pair_once_and_never_relabels` guards it.

First result: the obvious rules backfire. On the simulated instructor of Section 4 (a pool of 300 problems like the app's, held-out log-loss on 150 unseen problems, 8 seeds, refit every 5 labels), Figure 10(a):

- **Uncertainty sampling is harmful**, and it gets worse with more data: 0.634 at 100 labels and 0.650 at 300, against 0.570 and 0.548 for random. Its asked pairs have a mean true gap of only 0.77 logits (random: 1.69). Near a gap of 0 the instructor's answer is a coin flip, so uncertainty sampling diligently collects coin flips, and only along directions the model has already pinned down.
- **D-optimal design helps at first and then degrades**: 0.571 vs. 0.586 at 60 labels, but 0.557 vs. 0.548 at 300. Panel (b) shows why: one problem gets asked up to 20 times (random: 7). The determinant is shrunk fastest by *unusual* problems, the high-leverage points in feature space, so D-optimality keeps returning to the outliers.

Diagnosis: the acquisition function inherits the model's blind spot. Recall the planted truth $\delta(x) = w \cdot \varphi(x) + \tau\varepsilon(x)$. The part $\tau\varepsilon(x)$ belongs to the *problem*, and every comparison involving x shares it. The feature model has no term for it. It believes each comparison is fresh, independent evidence about θ, so it sees no harm in asking about the same outlier a fifth time. In fact the fifth comparison mostly measures that problem's own ε, and fitting it pulls θ the wrong way. Two quick experiments confirmed this. With $\tau = 0$ (no per-problem quirks) D-optimal design matched or beat random. With $\tau = 0.5$ it got worse with more data. This is the third time the same fact has mattered (Sections 5, 6, 8): *problems, not comparisons, are the independent units.*

The fix: put the blind spot into the model. Give every problem its own random effect,

$$\delta(x_i) = \theta \cdot z_i + u_i, \qquad u_i \overset{\text{iid}}{\sim} N(0, \tau^2),$$

a *mixed model* (fixed effects θ, random effects u). The design row becomes $a = [\, z_i - z_j,\ e_i - e_j \,]$ and the prior on u is a ridge penalty $1/\tau^2$ on those coordinates, so the Section 4 fitter works unchanged once it accepts a per-coordinate penalty:

From src/calcdiff/models/mixed_bt.py

```
G = np.hstack([self.Z_, np.eye(n)])                  # row i: g_i = [z_i, e_i]
D = G[pairs[:, 0]] - G[pairs[:, 1]]                  # a = g_i - g_j
```

```
prior = np.concatenate([np.full(d, self.l2), np.full(n, 1.0 / self.tau ** 2)])
self.fit_ = fit_newton(D, y, l2=prior)            # ridge = Gaussian prior, per coordinate
```

Now the design criterion should measure information about θ *alone*, since that is the part that generalizes to unseen problems. The joint covariance gets a rank-one update, and its θ-block becomes $\Sigma_{\theta\theta} - c\,vv^\top$ with $v = (\Sigma a)_\theta$. So, by the determinant lemma once more,

$$\mathrm{EIG}_\theta = -\tfrac{1}{2}\log(1 - c\,v^\top \Sigma_{\theta\theta}^{-1} v), \qquad \text{V-optimal:} \quad c\,v^\top C v.$$

When a problem's u_i is already pinned down, asking about it again still informs θ. When it isn't, most of the answer goes into u_i, and the formulas know that. Two tests check this: one against brute-force log-determinants of the inverted joint precision, and one showing that the mixed criterion reduces to the plain one as $\tau \to 0$.

Results. Combining the two ideas, V-optimal design (target the loss we are scored on) on the mixed model (respect the per-problem dependence), gives the blue curve in Figure 10(a): 0.577, 0.573, 0.548 and 0.541 at 60, 100, 200 and 300 labels, against random's 0.586, 0.570, 0.555, 0.548. It reaches random's 200-label loss with about 153 labels, roughly 23% fewer, while asking any one problem at most 13 times. Because every seed uses the same starting pairs and test set for every strategy, the honest test is the *paired* difference across seeds. V-optimal minus random is -0.007 ± 0.004 at 200 labels (lower in 6 of 8 seeds) and -0.007 ± 0.006 at 300 (lower in 6 of 8). The direction is consistent, but the effect is under two standard errors: *suggestive, not conclusive* at 8 seeds. The same paired test puts D-optimal at -0.014 ± 0.009 at 60 labels but $+0.012 \pm 0.012$ at 200, its early lead gone. The honest summary: uncertainty sampling clearly hurts (by an order of magnitude more than any other difference here), D-optimal design is a gamble, and V-optimal + mixed is a modest, steady improvement, worth roughly a fifth of the labels in the middle of the curve. Active learning is famous for large gains in settings with many parameters and cheap candidate pools. With 17 parameters, random pairs are already a decent design, and the main lesson is *not to make things worse*.

Replay on your labels. We can test on real data without new clicks. In each fold of the problem-level CV, the candidate set is the comparisons you already made among training problems, and "asking" reveals your stored answer. Panel (c) shows the result. At 60 labels, V-optimal + mixed gives held-out log-loss 0.332 (D-optimal 0.331) against 0.342 for random, and it matches random's 100-label loss with 80 labels. The effect is small but in the same direction. Replay understates the benefit, because the strategies can only choose among pairs that random sampling happened to produce.

In the app. The sidebar now has a *Pair selection* switch, defaulting to *active*. Each click refits the mixed model on all your labels (about 0.13 s) and asks the V-optimal pair, exploring at random 10% of the time. Repeats for measuring your consistency still appear. Each judgment now records how its pair was chosen (a new `strategy` column; the database migration marks all earlier judgments `random`, which is exactly what they were). That matters for analysis, which raises a subtle point.

Is it still legitimate to fit a model to adaptively chosen data? If pair k was chosen *because* of earlier answers, are the labels still a fair sample? For likelihood-based inference, yes, provided the model is correct. The design x_k depends only on past data $\mathcal{D}_{k-1}$, so the likelihood factorizes as $\prod_k P(y_k \mid x_k, \theta)\, P(x_k \mid \mathcal{D}_{k-1})$, and the second factor does not involve θ (Exercise 10.2). The design is *ignorable*. But if the model is wrong, adaptive designs magnify the error, because they sample

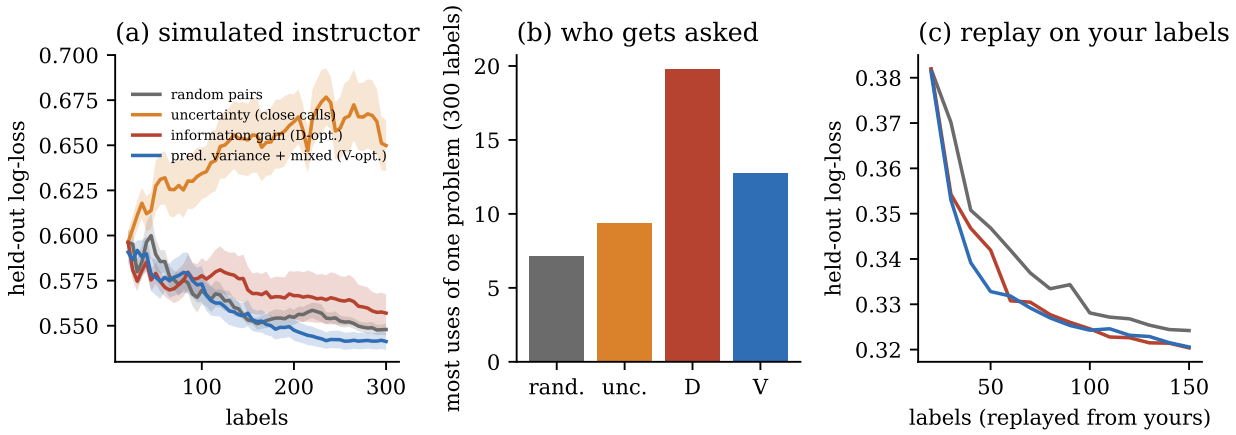


Figure 10: (a) Held-out log-loss vs. labels on the simulated instructor (mean ± s.e. over 8 seeds). (b) The most times any single problem was asked. (c) Replay on your labels, choosing only among comparisons you already made.

exactly where the wrong model is most interested. That is precisely what happened to D-optimal design above. Expect two practical side effects in your next session: closer calls (so more *About the same*, and slower answers), and a pool that is no longer a random sample of pairs, which is why Section 5's evaluations will keep using random comparisons on held-out problems.

Exercises.

Exercise 10.1. Prove the matrix determinant lemma $\det(A + uv^\top) = \det A\,(1 + v^\top A^{-1}u)$ and the Sherman–Morrison formula, and use them to derive both $\mathrm{EIG} = \frac{1}{2}\log(1 + ws^2)$ and the V-optimal score $c\,d^\top \Sigma C \Sigma d$.

Exercise 10.2. Let x_k be chosen by any rule depending only on $(x_1, y_1, \ldots, x_{k-1}, y_{k-1})$ and the learner's own randomness. Show that the likelihood of θ is proportional to $\prod_k P(y_k \mid x_k, \theta)$, exactly as if the x_k had been fixed in advance. Where does this argument break if the model family does not contain the truth?

Exercise 10.3. For the mixed model, show that if problem i has already been compared n_i times, the posterior variance of u_i is roughly $(1/\tau^2 + n_i\bar{w})^{-1}$. Use this to explain, without the full formula, why EIG_θ of yet another comparison involving i *increases* with n_i at first, as u_i gets pinned down, and why that is the opposite of what a naive "diversity bonus" would do.

11 A neural network that reads the expression tree

Every model so far saw a problem only through the 17 numbers of Section 3, and Section 3 called that choice the main source of bias. Two different problems with the same features (like $\cos(x^2)$ and $\sin(x^2)$) were indistinguishable. This section removes the middleman. A neural network reads the expression tree itself and *learns* its own representation. This is *representation learning*, the core idea of deep learning.

Learned structural recursion. Section 2 computed derivatives by one rule per node type. Here we compute an *embedding* $h(\text{node}) \in \mathbb{R}^{16}$ the same way, except that the rules are learned matrices:

$$\begin{aligned} h(x) &= \text{a learned vector}, \\ h(u^n) &= \tanh\big(W_{\text{pow}}[h(u);\, n;\, \mathbf{1}_{n<0};\, \mathbf{1}_{n\notin\mathbb{Z}}] + b\big), \\ h(u_1 + \cdots + u_m) &= \tanh\Big(W_{+} \textstyle\sum_k h(u_k) + b\Big), \\ h(u/v) &= \tanh\big(W_{\div}[h(u); h(v)] + b\big), \\ h(g(u)) &= \tanh\big(W_g\, h(u) + b_g\big) \qquad (g = \sin, \ldots, \sqrt{}), \\ \hat{\delta}(x) &= v \cdot h(\text{root}), \end{aligned}$$

and similarly for products and constants. This is a *recursive neural network* (Socher et al. 2011, and Tree-LSTMs, Tai et al. 2015). Notice how the algebra is encoded. Children of $+$ and $\times$ are *summed*, so the embedding is invariant under reordering summands (`test_sum_is_order_free_but_quotient_is_not`). Children of $\div$ are *concatenated*, so numerator and denominator stay distinct. Summing is no real restriction: by the Deep Sets theorem (Zaheer et al. 2017), every continuous permutation-invariant function of a bounded-size set has the form $\rho(\sum_k \phi(u_k))$. Summing rather than averaging matters too. The sum can count (two chain rules give a different input than one), while the mean cannot (Exercise 11.2). In all, the network has 3,104 parameters.

The code: compiling a forest into one batched computation. Evaluating trees node by node in Python would make thousands of tiny matrix products per pass. Instead, `compile_forest` merges all trees into one DAG, sharing identical subtrees since nodes are hashable (the 5,000 pretraining problems have only 13,713 distinct nodes). It sorts the nodes by height and then by type. Each (height, type) group becomes one contiguous block of rows computed by a single batched matrix product:

From src/calcdiff/models/tree_net.py

```
for groups in forest.levels:                        # height 0, 1, 2, ...
    H = torch.cat(blocks) if blocks else None       # embeddings of everything lower
    for g, start, stop, child, parent, extra in groups:
        if g == "pow":
            out = torch.tanh(self.pow(torch.cat([H[child], extra], dim=1)))
        elif g in ("add", "mul"):                   # sum the children of each node
            summed = torch.zeros(m, self.dim).index_add(0, parent, H[child])
            out = torch.tanh((self.add if g == "add" else self.mul)(summed))
        elif g == "div":                            # (numerator, denominator) side by side
            out = torch.tanh(self.div(H[child].reshape(m, 2 * self.dim)))
        ...
        blocks.append(out)
return torch.cat(blocks)[forest.roots]
```

`H[child]` gathers the children's rows. `index_add(0, parent, ...)` adds each child's row into its parent's slot, so a sum over a variable number of children is one vectorized call. Because children always have smaller height, their rows already exist when a level is computed. A forward *and* backward pass over 5,000 trees takes about 35 ms. A test compares the batched result with a plain recursive evaluation, node by node, for 60 random trees.

Backpropagation is reverse-mode differentiation. Training needs $\nabla_\Theta L$ for all 3,104 parameters. Section 2's dual numbers (*forward mode*) push one tangent direction through the computation per pass, so they would need 3,104 passes. *Reverse mode* pushes *adjoints* $\bar{v} = \partial L/\partial v$ backwards

from the scalar loss instead, and gets the whole gradient in one backward pass costing a small constant times the forward pass (the "cheap gradient principle", Baur–Strassen). For one node $h = \tanh(Wc + b)$ with c the children's input, the chain rule gives

$$\bar{a} = \bar{h} \odot (1 - h^2), \qquad \bar{W} \mathrel{+}= \bar{a}\, c^\top, \qquad \bar{b} \mathrel{+}= \bar{a}, \qquad \bar{c} = W^\top \bar{a}.$$

The adjoint $\bar{c}$ is passed to the children, and the "+ =" matters twice. A weight matrix is shared by every node of its type, and a subtree shared between trees in the DAG receives adjoints from several parents. In both cases the multivariate chain rule *adds* the contributions. In the code this is all the single line `loss.backward()`. PyTorch records the forward computation and replays it backwards. Since $|\tanh'| \le 1$, adjoints shrink as they pass through many levels. That is the *vanishing gradient* problem, which Tree-LSTMs and residual connections were invented to fix. Our trees are shallow enough not to need them.

From scratch, it memorizes. Training uses the same Bradley–Terry loss as everything else (`bt_loss`), full-batch AdamW, and early stopping on a held-out fifth of the training *problems*. Figure 11(c) shows one fold of your labels: 218 training comparisons for 3,104 parameters. Validation loss is best at epoch 8 (0.681, barely better than a coin's 0.693). After that, training loss keeps falling to 0.137 while validation loss climbs to 1.189. The network has memorized your answers. Out-of-fold, the scratch network reaches only 0.676 accuracy. Changing the learning rate, weight decay or width did not help. The simulation (panel b) shows how data-hungry it is: even with 10,000 comparisons it stays far behind the linear model (0.641 vs. 0.509).

Pretraining on free labels. Here is the asset we have not used yet. The generator makes unlimited problems, and the differentiator computes facts about each one at no cost. So before seeing any human judgment, train the encoder to predict those facts, the 17 quantities of Section 3, from the raw tree:

From src/calcdiff/models/tree_net.py

```python
def pretrain_encoder(trees, targets, dim=16, epochs=300, lr=0.01, seed=0):
    forest = compile_forest(trees)
    T = torch.tensor((targets - targets.mean(0)) / (targets.std(0) + 1e-9), dtype=torch.float32)
    enc, head = TreeEncoder(dim), nn.Linear(dim, targets.shape[1])
    opt = torch.optim.Adam(list(enc.parameters()) + list(head.parameters()), lr=lr)
    for _ in range(epochs):
        opt.zero_grad()
        loss = ((head(enc(forest)) - T) ** 2).mean()     # regress every target at once
        loss.backward()
        opt.step()
```

This is *self-supervised* pretraining in miniature, the same move that underlies large language models: learn a representation from abundant data with automatically computed targets, then adapt it to the scarce task. Pretraining on 5,000 problems takes 15 s (panel a). On *new* generated problems, a linear readout of the embedding recovers derivative size with $R^2 = 0.94$, chain-rule count with 0.96, quotients with 0.95, and the interaction `chain_in_product` with 0.81. The hardest target is *n_neg_exp* ($R^2 = 0.68$). The network has learned to *compute* the features from the tree.

Fine-tuning, and how not to. How should the pretrained encoder be used on your labels? There are three options, all in Table 7:

- **Frozen**: keep the encoder fixed and learn only the 16 weights of the head. This is a 16-parameter linear model on learned features, and it nearly matches the hand-made ones (0.332 vs. 0.310 log-loss).
- **Naive fine-tuning**: train everything at a small learning rate. This makes things *worse* (0.420). The head starts random, so the first gradients flowing into the encoder carry no information, and they degrade the pretrained representation before the head has learned anything (Kumar et al. 2022). Lowering the learning rate did not help.
- **LP-FT** (linear probing, then fine-tuning): first fit the head with the encoder frozen, then unfreeze everything at a small learning rate. The second stage starts from a sensible head, so its gradients are meaningful.

From src/calcdiff/models/tree_net.py

```
def fit_lp_ft(self, pairs, y, ft_lr=1e-3, rng=None):
    self.freeze_encoder = True
    self.fit(pairs, y, rng=...)                  # 1. linear probe: only the head moves
    self.freeze_encoder, self.lr = False, ft_lr
    self.fit(pairs, y, rng=...)                  # 2. fine-tune everything, gently
```

model	accuracy	log-loss
linear, 17 features	0.897	0.310
tree net, from scratch	0.676	0.622
pretrained, frozen	0.897	0.332
pretrained, naive fine-tune	0.879	0.420
pretrained, LP-FT	0.932	0.297

Table 7: Out-of-fold results on your 337 judgments (5-fold CV over problems, 3 repeats).

LP-FT gives the best numbers in these notes so far: log-loss 0.297 against 0.310 for the linear model. The paired problem-bootstrap interval for the difference is $[-0.052, 0.025]$, though. On *NNJudg* judgments, a network that reads raw trees *ties* a model built on expert-designed features. In simulation (panel b), pretraining+LP-FT is better than the linear model with 100 comparisons (0.519 vs. 0.533) and equal with 10,000 (0.512 vs. 0.509; the oracle is 0.465).

What to make of a tie. Be honest about what the pretrained network knows. Its pretraining targets *were* the hand-made features, so it learned what the feature designer already knew and then matched the feature model. The gains from learned representations show up when (i) the useful structure is something nobody thought to make a feature (Exercise 11.3), or (ii) there is far more labeled data than *NNJudg* judgments. The real lessons of this section are about the machinery: reverse-mode differentiation through a recursive structure, how quickly a few thousand parameters memorize a few hundred labels, and how to use cheap data you have in abundance (generated problems) to make expensive data you have little of (your judgments) go further.

Exercises.

Exercise 11.1. Write out reverse mode for a two-level tree $h_{\text{root}} = \tanh(W_{\text{sin}} h_1 + b)$, $h_1 = \tanh(W_{\text{pow}}[h_x; n; \dots] + b')$, with loss $L = \ell(v \cdot h_{\text{root}})$. Show that computing $\nabla_\Theta L$ costs a constant multiple of evaluating L, while forward mode would need one pass per parameter.

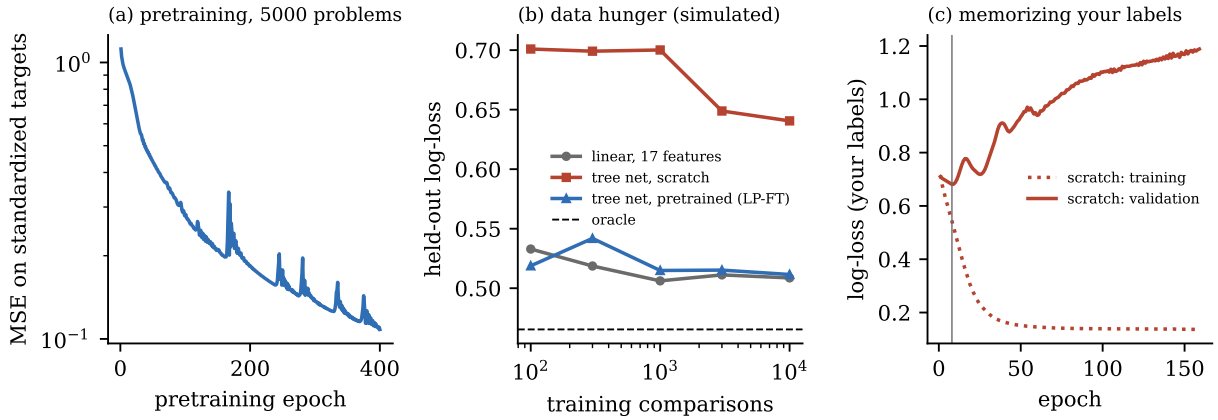


Figure 11: (a) Pretraining loss on generated problems. (b) Held-out log-loss vs. training comparisons on the simulated instructor. (c) One fold of your labels: the scratch network's training and validation loss; the vertical line marks early stopping.

Exercise 11.2. Show that a mean aggregator $\frac{1}{m}\sum_k h(u_k)$ gives $u+u$ and u (as summands) the same input, while a sum does not. Which features of Section 3 could a mean-aggregating network not represent?

Exercise 11.3. Pretraining targets are free to choose. Propose a target that the differentiator can compute but that is *not* among the 17 features (for example, the number of terms in the *simplified* derivative, or whether simplification cancels anything). Argue why a network pretrained on it might beat the feature model on your labels, and how you would test it.

12 Explaining a prediction: Shapley values

The model says A is harder than B with probability 0.94. *Why?* Section 9's split gain answered a global version of that question badly. It credits whichever of several correlated features a tree happened to split on. This section answers the local question, for one comparison, with the only attribution rule that satisfies four natural axioms. The answer also becomes the explanation the guessing game of Section 13 shows players.

A comparison as a cooperative game. Let f map a feature vector to a predicted difficulty, and fix two problems with features a, b. Partition the 17 features into $k = 7$ groups that mean one thing to an instructor (size, chain rule, quotient rule, product rule, function types, tricky exponents, powers and constants). These groups are the *players*. A coalition S of groups "switches on" A's values for its groups and leaves the rest at B's:

$$v(S) = f(a_S, b_{\bar{S}}) - f(b), \qquad v(\varnothing) = 0, \qquad v(\text{all}) = f(a) - f(b).$$

So v interpolates between the two problems, and the question is how to divide the total gap $f(a) - f(b)$ among the groups.

Theorem 12.1 (Shapley 1953)**.** *There is exactly one map $v \mapsto \phi(v) \in \mathbb{R}^k$ satisfying (i)* efficiency*: $\sum_j \phi_j = v(all)$; (ii)* symmetry*: if $v(S \cup j) = v(S \cup j')$ for all $S \not\ni j, j'$, then $\phi_j = \phi_{j'}$; (iii)* null

player: *if* $v(S \cup j) = v(S)$ *for all* S*, then* $\phi_j = 0$*; (iv)* additivity*:* $\phi(v + w) = \phi(v) + \phi(w)$*. It is*

$$\phi_j = \sum_{S \not\ni j} \frac{|S|!\,(k - |S| - 1)!}{k!} \big[v(S \cup j) - v(S)\big],$$

the marginal contribution of j *averaged over all* $k!$ *orders in which the players could arrive.*

Proof sketch. Games on k players form a vector space of dimension $2^k - 1$. The *unanimity games* $u_T(S) = \mathbf{1}[T \subseteq S]$, $T \neq \varnothing$, are a basis: Möbius inversion writes any $v = \sum_T a_T u_T$ with $a_T = \sum_{S \subseteq T} (-1)^{|T|-|S|} v(S)$ (the *Harsanyi dividends*). On $c\,u_T$, players outside T are null, so get 0 by (iii), and those inside are symmetric, so by (i) and (ii) each gets $c/|T|$. This holds for every real c, so additivity determines $\phi(v) = \sum_T \phi(a_T u_T)$ uniquely. Checking that the permutation formula satisfies (i)–(iv) is direct. □

Efficiency is what makes Shapley values right for *pairs*. With B as the baseline, the attributions add up exactly to the model's predicted gap, so they literally decompose the log-odds $\hat{\delta}(A) - \hat{\delta}(B) = \log \frac{P(A \succ B)}{P(B \succ A)}$. For a *linear* model the game is additive, and the formula collapses to $\phi_j = \sum_{i \in G_j} \theta_i (a_i - b_i)$ (Exercise 12.1). For the boosted model of Section 9 nothing collapses, and we compute the formula exactly.

Exact computation in one vectorized call. With $k = 7$ there are $2^k = 128$ coalitions. Build all 128 hybrid feature vectors at once, score them in a single call, and read off every marginal contribution:

From src/calcdiff/explain.py

```
masks = _masks(k)                                   # (2^k, k): row r = binary digits of r
X = np.repeat(b[None, :], len(masks), axis=0)       # start every coalition at B's features
for j, cols in enumerate(groups):
    X[np.ix_(masks[:, j], cols)] = a[cols]          # members of the coalition take A's values
v = f(X) - f(X)[0]                                  # row 0 is the empty coalition
index = masks @ (1 << np.arange(k))                 # each coalition's row number
for j in range(k):
    without = ~masks[:, j]                          # all S not containing j
    with_j = index[without] | (1 << j)              # S + j, by setting bit j
    phi[j] = np.sum(weight[size[without]] * (v[with_j] - v[without]))
```

Representing coalitions as the bits of an integer makes "$S \cup \{j\}$" a bitwise OR, so the whole sum is array indexing. Over 400 random pairs, $\sum_j \phi_j$ matched the model's gap to within $2e - 15$, which is efficiency up to floating-point rounding. Tests check every axiom: efficiency on a boosted model, null player (a group the model ignores gets exactly 0), symmetry (a pure interaction $x_q x_p$ is split $\frac{1}{2}, \frac{1}{2}$), and the linear closed form.

When 2^k evaluations are too many, average marginal contributions over n random orders instead (`shapley_sampling`). The estimate is unbiased, and each single order already satisfies efficiency exactly. Its error falls like $n^{-1/2}$. Figure 12(c) measures a slope of -0.48, and 20 orders already give an RMS error of 0.033 logits.

Why groups? Section 6 showed that ridge spreads weight across near-duplicate features (`n_chain` vs. the function counts). Per-feature attributions inherit that arbitrariness. Making the whole family one player sidesteps the question of how to split credit inside it, a question the data cannot answer. It also makes the explanation say something an instructor recognizes. And it cuts the cost from 2^{17} to 2^7 evaluations.

What drives your model's judgments. Figure 12(a) averages $|\phi_j|$ over 400 random pairs of pool problems for the two models fitted to your 346 judgments. For linear BT the largest is the *chain rule* group (30% of the total), then *size*. For boosted BT it is *size* (61%), then *chain rule*. The rankings broadly agree (Spearman 0.93), but the two models, which predict almost equally well (Sections 8 and 9), divide the credit quite differently. Breiman called this the *Rashomon effect*: many models fit the data equally well and tell different stories. So Shapley values explain *the model*, not your mind. When near-duplicate features exist (chain-rule counts and size move together), the data cannot tell which story is true. Split gain is more lopsided still. It gives *size* 77% of all gain and ranks the quotient group 6th of 7, because it rewards features that are split on often, not features that move predictions.

Explaining single comparisons. `describe` turns ϕ into the sentence a game player will see. It is written from the side of the problem the model judges harder: if $\sum_j \phi_j < 0$, it negates ϕ first. My first version forgot this, and for a pair where the model preferred B it cheerfully explained what made A harder.

From src/calcdiff/explain.py

```
def describe(phi, names=None, top=2):
    if phi.sum() < 0:                  # the model picked B: justify B
        phi = -phi
    order = np.argsort(-phi)
    up = [names[j] for j in order[:top] if phi[j] > 0.05]
    down = [names[j] for j in order[::-1][:1] if phi[j] < -0.05]
    ...
    s = "Mostly the " + " and the ".join(up)
    return s + (f", despite the {down[0]}" if down else "") + "."
```

Table 8 explains three of your comparisons with the final linear model: a confident one, a close call, and one where the model disagrees with you. Panel (b) draws the first as a waterfall. The gap of 2.67 logits ($P = 0.94$) is built mostly from the quotient rule, with the function types pulling back. That first row also holds a trap. $\frac{\sqrt{4x^3}}{2x^{3/2}}$ *simplifies to* 1 for $x > 0$, so a student who simplifies first does almost no work. The model credits the quotient rule because none of its 17 features can see "this cancels". That is exactly the kind of pretraining target Exercise 11.3 asks for, and it is a good illustration of what explanations are for: they make a model's blind spots visible.

A caution about hybrids. The coalition $v(S)$ evaluates the model on *hybrid* feature vectors: A's chain-rule counts with B's size, say. Some hybrids correspond to no real problem (three nested chain rules in a five-node tree). The model is then extrapolating, and so is the explanation. Conditional variants of Shapley values avoid this by averaging over realistic completions, but they need a model of the feature distribution and break the null-player axiom. For explaining a *comparison*, where both endpoints are real problems, the baseline version used here is the standard choice.

Exercises.

Exercise 12.1. Show that for $f(x) = \theta \cdot x + c$ the game is additive, $v(S) = \sum_{j \in S} \sum_{i \in G_j} \theta_i (a_i - b_i)$, and deduce the closed form. Is the converse true: does $\phi_j(a, b) = \sum_{i \in G_j} \theta_i (a_i - b_i)$ for all a, b force f to be linear?

Exercise 12.2. Compute the Harsanyi dividends of $v(S) = f(a_S, b_{\bar{S}}) - f(b)$ for $f(x) = x_1 x_2 x_3$ with $a = (1, 1, 1)$, $b = (0, 0, 0)$, and verify the Shapley values $\frac{1}{3}$ each. What do the dividends of a boosted model of depth D look like, and why do they vanish for coalitions larger than D?

you said harder	than	P	why, according to the model
$x^{-1} + x^{1/2} + \dfrac{\sqrt{4x^3}}{2x^{3/2}} + 2$	$x^4 + x^2 + \tan\left(x^4 - 4\right)$	0.94	quotient rule +1.7, tricky exponents +0.9, function types −0.8 *Mostly the quotient rule and the tricky exponents, despite the function types.*
$5x + \left(e^{\ln\left(x^{-2}+5\right)+8} - 1\right)^5$	$e^{\cos\left(x^2+8\right)x^2} + 9x^{1/2}$	0.58	chain rule +0.7, powers and constants −0.4, size +0.3 *Mostly the chain rule and the size, despite the powers and constants.*
$x^2 + \left(\left(x^{1/3} + 3\right)^4 - 3\right)^2$	$x^3 + 7\left(-4x^{1/2}\right)\sin\left(x^{-1}e^{-5x^2+1}\right)$	0.08	size −1.6, function types −0.6, powers and constants +0.5 *Mostly the size and the function types, despite the powers and constants.*

Table 8: Shapley explanations of three of your judgments by the final linear model: the three largest group contributions (logits, positive = toward the problem you chose) and the generated sentence, which justifies the model's own pick. P is the model's probability that the problem you chose is harder.

Exercise 12.3. Construct two functions f, g on two binary features with $f = g$ on three of the four corners ("the data") but different Shapley values for the pair $a = (1, 1)$, $b = (0, 0)$. Relate this to the Rashomon effect and to the hybrid caution above.

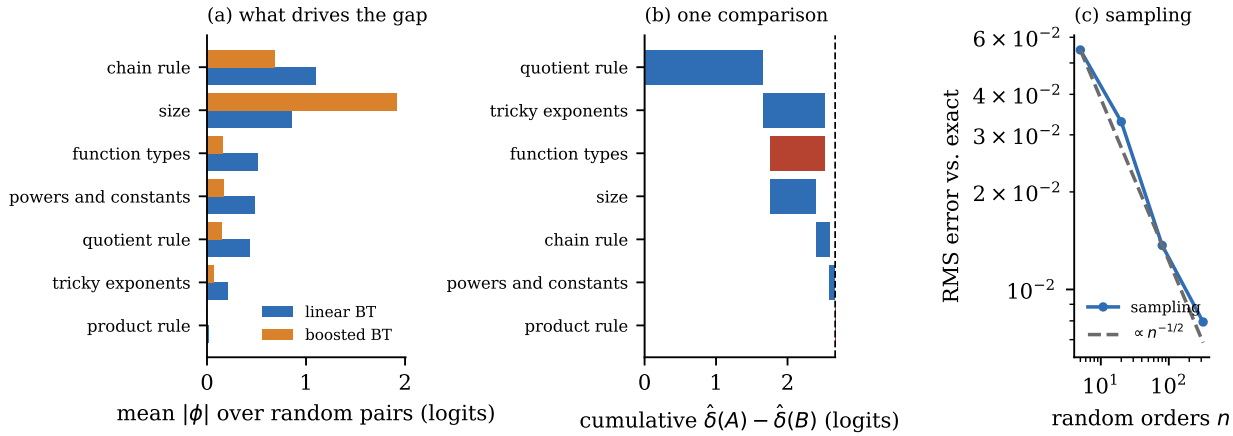


Figure 12: (a) Mean $|\phi|$ per feature group over random pairs, for linear and boosted BT fitted to your labels. (b) Waterfall for the first row of Table 8: contributions accumulate to the predicted gap (dashed). (c) RMS error of the sampling estimator against exact values, vs. number of random orders.